AEMaaCS

25 September 2026 09:59

Day 1: Sep 25th

AEM = Adobe experience manager

Definition : it is adobe's **content management platform** used to create ,manage , organize and deliver digital experiences.

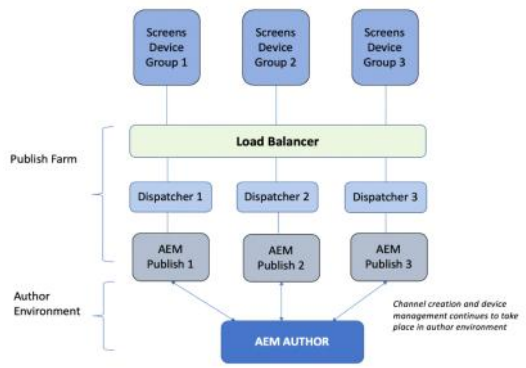
Popular products

AEM sites	Used to create and manage web pages
AEM assets	Used to manage digital assets(images ,vid ,pdf ,doc ,logo)
AEM headless	Instead of AEM directly rendering website , AEM can provide content through API

AEMaaCA
Adobe Experience Manager as a Cloud Service

It is the cloud native version of AEM operated through adobe's cloud infrastructure.

Architecture:



USER
|
CDN
|
Dispatcher
|

| |
Publish instance Publish instance
| |

|
Content replication/deployment
|
Author Instance

Author:

Author environment is where content authors work.
Author is where content is created and managed
Eg: a marketing employee wants to create
"Summer Sale - 30% Off"
They log into AEM Author and create/edit the page.
They can :

- Create pages
- Edit content
- Add component
- Upload assets
- Configure components
- Manage navigation
- Preview content
- Publish content

Publish:

The publish environment contains content that is intended to be delivered to website users.

After marketer publish that content becomes available through publish.

Dispatcher:

Dispatcher sits between the external request and the publish environment.

User
|
CDN
|
Dispatcher
|
Publish

Dispatcher provides functions such as

- Catching
- Request filtering
- Security-related request handling
- Load distribution in appropriate architecture

Why catching?

Suppose 10000 people request
/luma/products/shoes

Instead of asking publish every single time:
Dispatcher can serve cached content

CDN:

Content Delivery network

This is closer to the user geographically and helps deliver cached content efficiently.

Think:
User in Bangalore
|
CDN edge
|
AEM infrastructure

The complete request flow:

Suppose I enter: https://www.luma.com/shoes

Conceptually:
Browser
|
CDN
|
Dispatcher
|
Publish
|
AEM content
|
Response
|
Browser

If the content is already cached:

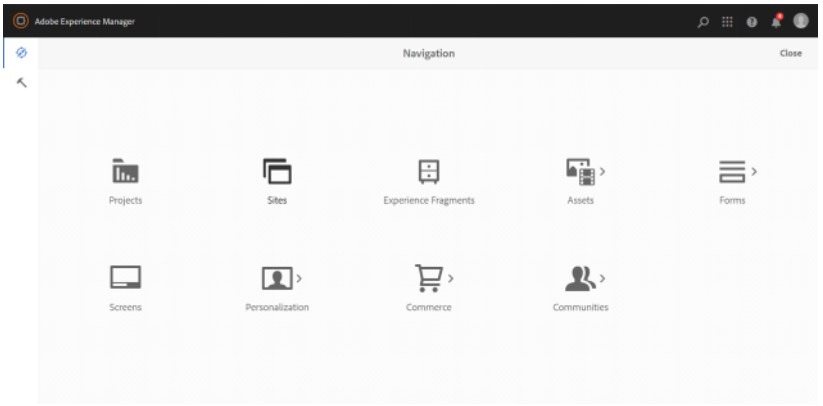
Browser
|
CDN/dispatcher
|
Cached content
|
Browser

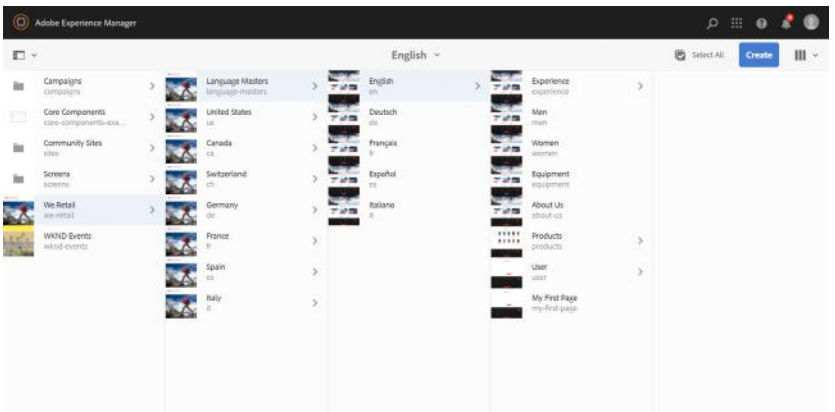
Hands-0n (AEM)

1.Create a Page

Open Author instance - java -jar aem-author-p4502.jar

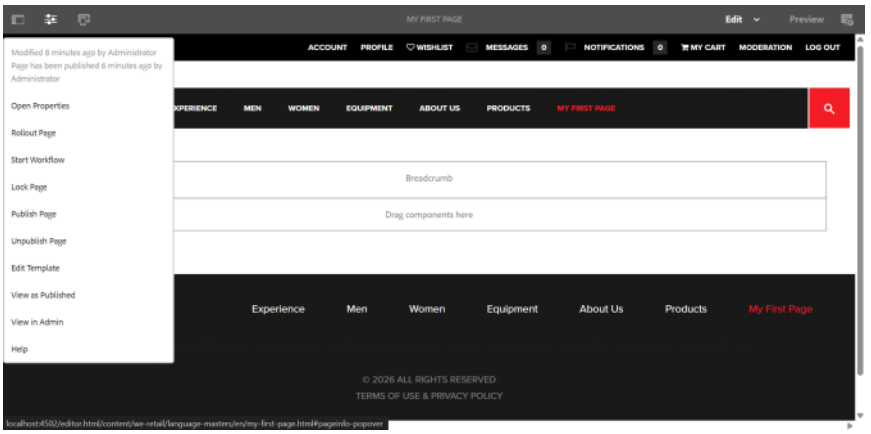
Navigation -> sites -> we. Retail -> English -> Create





Select the Template -> content page (Eg) - give a Title -> click create

Page will be created
If you need to publish and see in publish instance
Go to-> PAGE INFORMATION tab --|_-



Click publish page and it will be published and available in publish instance in port 4503
http://localhost:4503/content/we-retail/language-masters/en/my-first-page.html

2. Uploading a sample package to author instance

Upload a package -> selected the downloaded package zip
Install

3. Transferring package from Author to publish instance

Create a page(refer hands-on 1) with name package manager example
Go to tools -> CRXDE lite -> Package Manager tab -> create package

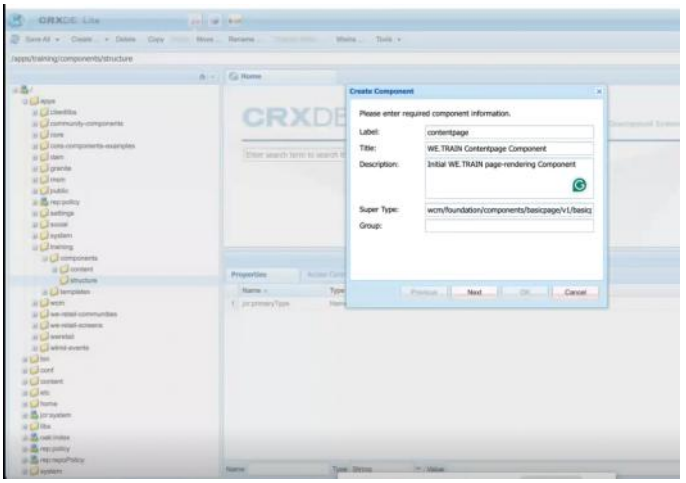
Name package
Add filter -> browse the page we created
content -> dam -> language master -> eng-> package manager example

Build it
Download it

Now go to publish instance 4503
Upload a package -> selected the downloaded package zip
Install

4. Create page rendering component

Inside CRDX create a folder inside app
Training
- -components
  - -content
  - -structure -> create component
- -templates



Click next and next
It will create a default rendering component with same name as component(contentpage)

The default rendering script in Adobe Experience Manager (AEM) is a file that matches the exact name of the component (such as contentpage.html or contentpage.jsp) and handles generating the output when no specific selector or alternative script is matched

It will be in JSP (javaserver pages) - It is used to create dynamic, data-driven web applications. It is used to create dynamic, data-driven web applications.

Rename it to .html
And add sample html code

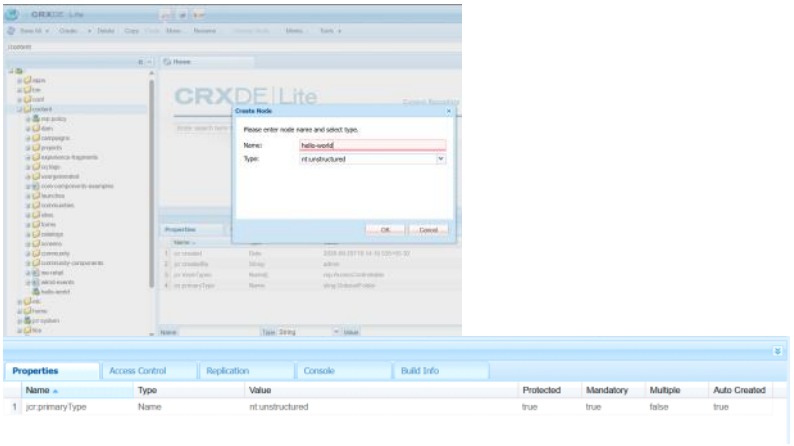
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Webpage</title>
</head>
<body>

  <h1>Welcome to My Website</h1>
  <p>This is a basic paragraph of text on a simple HTML page.</p>

  <!-- This is a comment. It will not be visible on the browser. -->

</body>
</html>
```

Now in root content folder - right click -> create node - name(hello-world) (note : not in training/component/content)



At the bottom add a property of

Name	Sling:resourceType
Type	String
Value	/apps/training/components/structure/contentpage

And save

So

-- when we request a URL

Eg: http://localhost:4502/content/hello-world.html
sling takes URL as req ,decompose URL into various sections

protocol->http
host -> localhost
port -> 4502
content path-> /content/hello-world (note path will begin right after port number and end before the extension)
extension- html

based on which it goes to particular server and content path
and figure out the rendering script which is not inside the /content

sling:resourceType points to the location of rendering script

so it goes to /apps/training/component/structure/contentpage

and give it back to browser.

For every URL request, the following steps are performed to get it resolved:

1. Decompose the URL
2. Search for a servlet or a vanity URL redirect
3. Search for a node indicated by the URL
4. Resolve the Resource
5. Resolve the rendering script/servlet
6. Create rendering chain
7. Invoke rendering chain

Decomposing the URL

Consider the following example:
URL: http://myhost/tools/spy.printable.a4.html/a/b?x=12

The above URL can be decomposed into the following components:

protocol	Host	Content path	Selector(s)	Extension		Suffix		Param(s)
http://	myhost	tools/spy	.printable.a4	html	/	a/b	?	x=12

Selector:

Selects particular rendering script rather than default one
Select is between first bot and last bot
http://localhost:4502/content/hello-world.blue.html'

So here blue is the selector so sling search for blue rendering script inside app

If there is multiple selector
Eg: http://localhost:4502/content/hello-world.blue.red.html

Preference is given to first selector

Which is blue

If first selector fails to load default script is loaded

Sling resource resolution = content rendering

In JCR everything is node and properties
In sling everything is resource

Not every node is called resource
For a node to become a resource it should fulfill 3 conditions
1.node should have a unique name
2.node should have a unique path
3.node should have or able to resolve sling:resourceType property

Microkernel
TarMK
File system -> Tar files

mongoMK
Database(MongoDB) -> json files

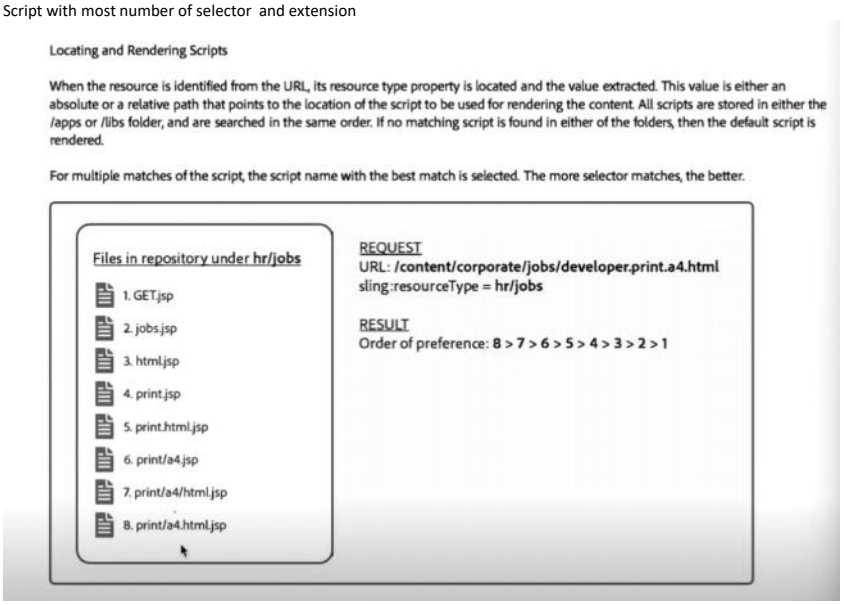
An absolute path is the complete location of a file or folder starting from the absolute root of the file system,

while a relative path is a shortcut that points to a location *relative* to your current working directory

Refer session_6_userGuid.pdf

Order of Preferring

Script with most number of selector and extension



Type of Templates

1.static -one time blue print copy where a page is created ,
Note: later changes made to template do not affect existing page
-deprecated in modern AEM
-it is defined directly by developers in repository code.
2.Dynamic(Editable)-
Modern best practice in AEM
Allowing authorized content authors and administrators to create and modify page layout through UI.

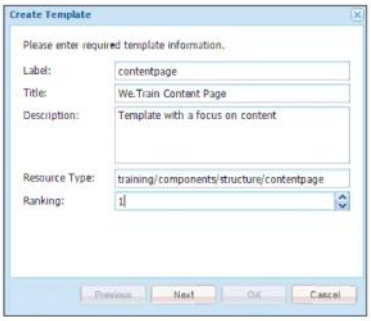
Creating a Template

Exercise - I

Templates play many parts within the AEM ecosystem. One important part is to be in the list of available templates when the content author creates a page.

Task – Create a Template

1. **In CRXDE Lite, navigate to /apps/training/templates.**
2. **Right click /apps/training/templates and select Create...> Create Template.**
3. **Enter the following values:**
   - **Label**: contentpage
   - **Title**: We.Train Content Page
   - **Description**: Template with a focus on content
   - **Resource Type**: training/components/structure/contentpage
   - Ranking: 1



4. **Click Next.**
5. **Click the + and enter /content(/.*)? for Allowed Paths.**

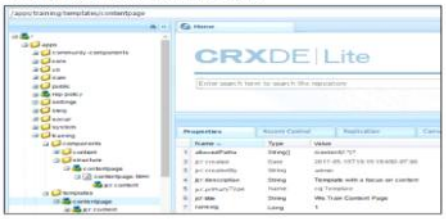


6. **Click Next.**
   NOTE: Do not click on the "+" for Allowed Parents.

4

7. **Click Next.**
   NOTE: Do not click on the "+" for Allowed Children.
8. **Click OK.**
9. **Save all changes.**

Your project node structure should look like this:



Restricting the access to template

Modify the value of the allowed Paths property (by double-clicking on this property) to be: /content/we-train(/.*)?

Task – Restrict Template Use

You can restrict where a template can be used in two ways. We have already seen that the allowedPaths property on the /apps/training/templates/contentpage template will define the paths where this template can be used to create pages. This restricts the template from the template's point of view.

1. **Navigate to the AEM Sites Console. Click Create Page.**
2. **Notice that the We.Train template appears.**
3. **Do the same with Sites>Screens. Notice the We.Train template appears.**

NOTE: We probably want the We.Train template to appear as an available template only for the We.Train site.

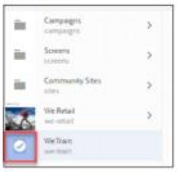
4. **Using CRXDE Lite, navigate to /apps/training/templates/contentpage.**
5. **Modify the value of the allowedPaths property (by double-clicking on this property) to be:**

   /content/we-train(/.*)?

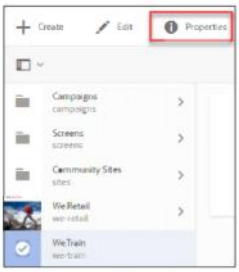
6. **Repeat steps 1-3. Notice that this time the We.Train template no longer appears as a choice for Screens.**
7. **Navigate to We.Train.**
8. **Click Create Page. Notice that the We.Train Content Page template is available.**

Adding thumbnail to template

1. Using the Sites Console, navigate to We.Train.
2. Select We.Train:



3. Click Properties.



4. Click the Thumbnail tab.



5. Click on Upload Image and upload We-train.png from the exercise files provided to you for this exercise into the thumbnail.

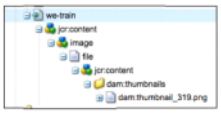


6. Click Save & Close in the upper right.

How to make this thumbnail available for every page created from that template has this thumbnail.

9. Using CRXDE Lite, navigate to /content/we-train/jcr:content.

Notice the newly defined structure under the jcr:content node. **NOTE:** You may need to refresh CRXDE Lite.

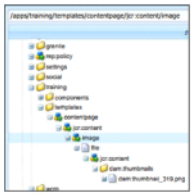


10. Right-click on the image node and select Copy.

NOTE: Do not copy the "dam:thumbnail_319.png" node. Instead, ensure you are copying the image node directly under jcr:content.

11. Navigate to /apps/training/templates/contentpage/jcr:content.

12. Right-click the jcr:content node and select Paste. Your node structure should now look like this (NOTE: The "319" number at the end of the *.png filename may be different in your instance):



HTL - HTML Template language
Server side templating system used in AEM for building html pages

SYNTAX:

1. Expressions (${...})

Expressions are used to output dynamic data, read variables, or pull content from dialog properties

They provide automatic context-aware escaping to prevent Cross-Site Scripting (XSS) vulnerabilities

Accessing Dialog Properties:	<h1>${properties.title}</h1> <p>${properties.description}
String Concatenation & Options (using **@**)	${'Page title is: {0}' @ format=[currentPage.title]}

2. Block Statements (data-sly-*)

Block statements are custom attributes prefixed with data-sly- added to standard HTML tags.

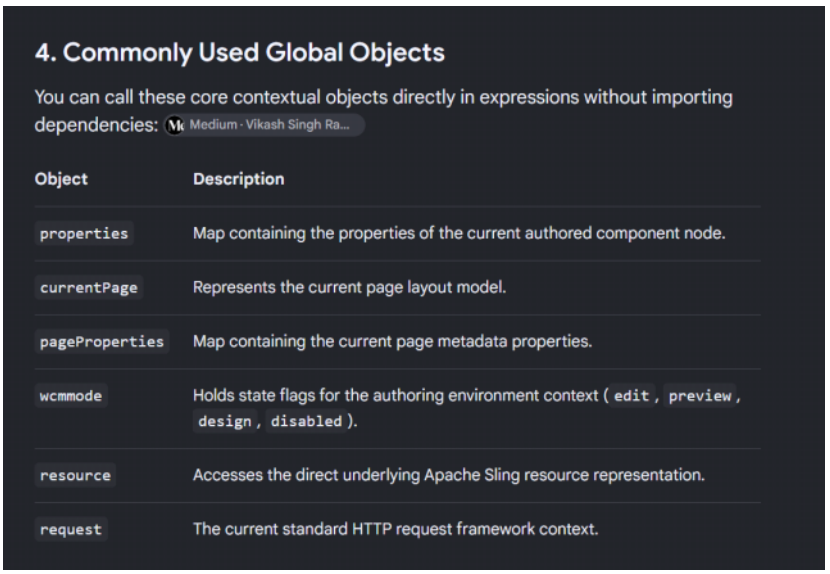
To inject logic without adding an extra wrapper HTML tag to the final markup, use the custom <sly> element, which drops itself from the final output.

Logic & Conditionals (**data-sly-test**) act as IF condition	This shows up only if true
IF else	<sly data-sly-test.hasLinks="${properties.linkUrl}"> Click here </sly> <sly data-sly-test="${!hasLinks}"> No link provided. </sly>
Iteration & Loops (data-sly-list or data-sly-repeat)	<ul data-sly-list="${currentPage.listChildren}"> ${item.title}
custominzing	<ul data-sly-list.child="${currentPage.listChildren}"> Index ${childList.index}: ${child.title}
Linking Backend Logic(**data-sly-use**)	<divdataslyuse.myModel= "com.example.core.models. CustomComponentModel"> ${myModel.authorName}
Including Resources	
Including Files	<footer data-sly-include="footer.html"></footer>
Dynamic Attributes	
data-sly-attribute: Adds attributes dynamically	<div data-sly-attribute.class="${properties.customClass}">
data-sly-element: Dynamically changes the HTML tag type	<h1 data-sly-element="${properties.headingLevel || 'h2'}">Dynamic Heading</h1>

3. HTL Comments

Standard HTML comments (<!-- comment -->) will render in the final browser source view.

Eg: <!--/* This comment is server-side only and will not show in final source HTML */-->



currentPage:

https://experienceleague.adobe.com/en/tools/aem-api-documentation/6-5/javadoc/com/day/cq/wcm/api/Page.html

CurrentNode:
https://experienceleague.adobe.com/en/tools/aem-api-documentation/spec/javax.jcr/javadocs/jcr-1.0/javax/jcr/Node.html

Session 9

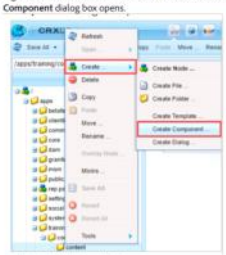
HTL Component

Create an HTL Component

Exercise 1 – Creation of a content component (HTL component).

In this exercise, you will create a simple component that will be added to a page on the We.Retail website.

1. In CRXDE Lite, navigate to **/apps/training/components/content**.
2. Right-click the **content** folder and click **Create** > **Create Component**, as shown. The **Create Component** dialog box opens.



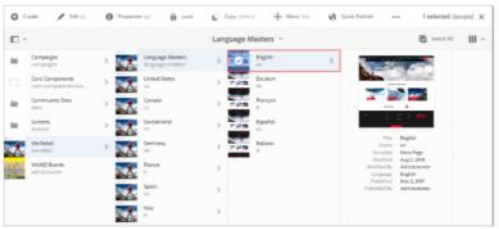
3. Enter the following details:
   a. In the **Label** field, enter **quote**.
   b. In the **Title** field, enter **Quote**.
   c. In the **Description** field, enter **My First Component**.
   d. Leave the **Super Type** field empty.
   e. In the **Group** field, enter **We.Retail** and click **Next**. The **Advanced Component Settings** are displayed.
   f. Leave the **Advanced Component Settings** blank.
   g. Click **OK**. The component is created.
   h. Click **Save All** to save the changes

NOTE: The We.Retail componentGroup was used so that the component can be added to the We.Retail website.

4. Expand the Quote component you created now by clicking the + icon next to it.
5. Right-click quote.jsp and click Rename to rename it.
6. Rename it as quote.html.
7. Double-click quote.html. The editor opens on the right.
8. Delete the existing content in quote.html and replace it with the code from start-quote.html from your exercises folder.
9. Click Save All to save the changes.

To create a dialog, where components can be added:

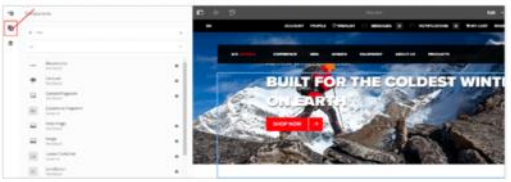
10. Right-click the quote component and click Create > Create Node. The Create Node dialog box opens.
11. In the Name field, enter cq:dialog.
12. In the Type field, select nt:unstructured if it is already not selected.
13. Click OK. The node is created
14. Click Save All to save the changes.
15. In the AEM author instance, navigate to Sites > We.Retail > Language Masters > English, as shown :



16. With the English page selected, click Edit (e) from the top menu bar to open the English page.
17. In the upper left, click the Toggle Side Panel icon, as shown, if you do not see the left panel.



18. In the left panel, click the Components icon, as shown. A list of available components is displayed:



19. Look for the Quote component and drag and drop the component, as shown, onto the layout container:



NOTE: Even though the component was created in the /apps/training code base, you were still able to see and use the component on the We.Retail website. This is because the property componentGroup is We.Retail on the /apps/training/components/content/quote node.

session 10

responsive grid component - path: content>men->jscr>root>responsive grid

static Template: code-defined blueprints where page structure is copied once at creation time
Created and managed strictly by developers through code and the JCR repository under /apps (using JSP/HTL). Any layout or structural change requires a full code deployment.

Editable Templates:page templates that specialized users called template authors can create and manage directly through the AEM UI without writing code.

however to enable this in AEM developers must create a required environment and create clint libraries and component to be used on the page.

Static Templates	Editable Templates
Are developed and configured by developers	Are created and edited by authors
Have the same structure as the page	Help define the structure, initial content, and content policies for pages
Are copied to create a new page and do not have a dynamic connection with the page	Maintain a dynamic connection between the template and pages
Use the Design mode to persist design properties	Use content policies (edited from the template editor) to persist the design properties
Are stored under /apps	Are stored under /conf

Hands-on
Exercise 1: Create an editable template

Task 1: Create a new page rendering structure component
go to training/structure -> create component

Task 1: Create a new page rendering structure component

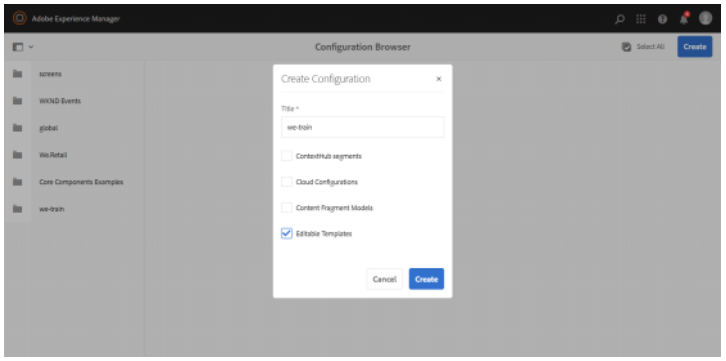
Before we create editable template, we need to create a new page/structure component which will inherit from the core component's page component.

1. Right-dick /apps/training/components/structure and select Create...> Create Component.
2. Fill in the following values:
   - **Label**: page
   - **Title**: WE.TRAIN page Component

- **Description**: WE.TRAIN page-rendering Component
- **SuperType**: core/wcm/components/page/v2/page

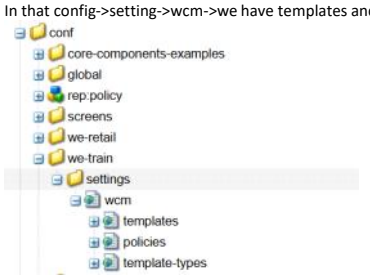
3. Click Next.
4. Click OK, and save your changes.
5. Right-dick on page.jsp to rename it to page.html. Click on Save All.
6. Now your page component is ready to work with editable templates.

Task 2: Create an empty page template type
Go to Tools->config browser -> create



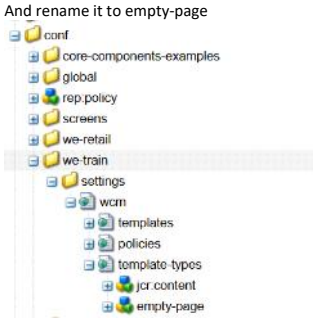
So there will be folder we-train created in config folder in CRXDE

In that config->setting->wcm->we have templates and template type



For a page there will be many templates but one template type
So will create a template type
Copy html5page node from libs/settings/wcm/template-type/html5page

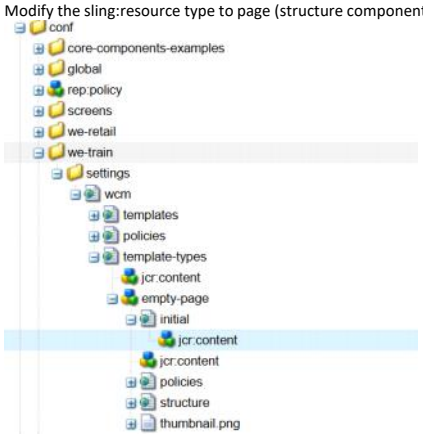
Paste that in template-type of conf->we-train->settings->wcm->template-types
And rename it to empty-page



Inside empty-page->jcr:content
Modify the property

a. **jcr:title: We.Train Empty Page**
b. **jcr:description: Empty Template for a We.Train page**

Next in
Initial->jcr:content
Modify the sling:resource type to page (structure component we created)



4	sling:resourceType	String	training/components/structure/page	false	false	false	false

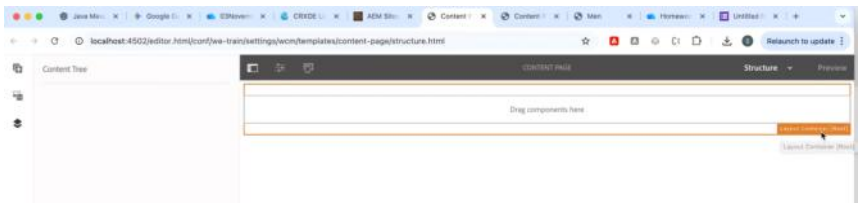
Also do same for template-types->structure->jcr:content

Navigate to empty-page/policies/jcr:content, select the root node, right-click it, and click Delete to delete the node. Now, you can start with an empty root layout container.

Delete the page.html in training /component/content/page.html

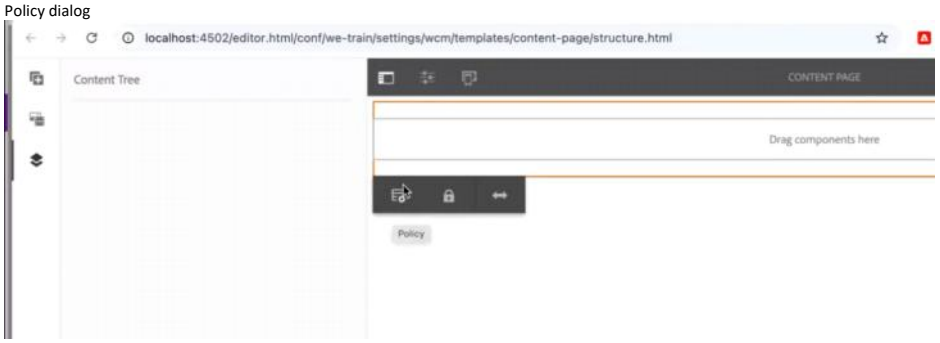
Then the option to drag the component appears in the template section content page

Task 3:Create a development template



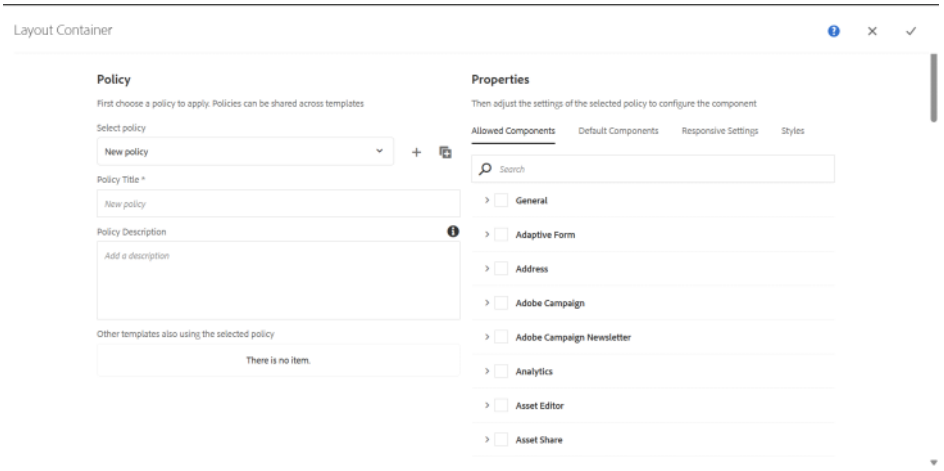
The orange border is the layout container(root) / root layout container
Root layout container can only be edited in template
Which is locked
Whichever is locked appears in structure mode

Policy dialog

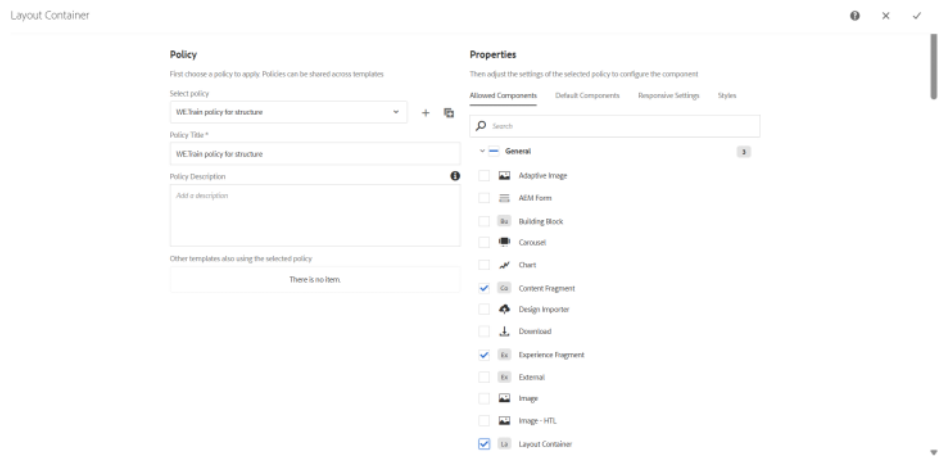


Where we can define the policy/properties for the layout container

Move the cursor to bottom orange NOT inside(drag components here)



Edit the policy title and properties allowing content fragment ,exp fragment,layout container



Create one more layout container and unlock it

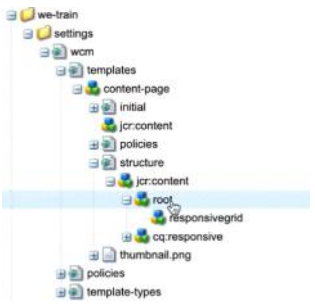
Root layout container which is locked on the template

Inside root layout container we dragged another layout container and we called it inner layout container and then we unlocked it. As soon as you unlock it, this inner layout container will allow you to edit content on the page.

Locked = Structure
Unlocked = Able to be edited on pages = Initial content

Unlocking makes it available in initial node also from structure node
Any change made in structure mode it sits in structure node only , when it is unlcoked it is also available in initial node

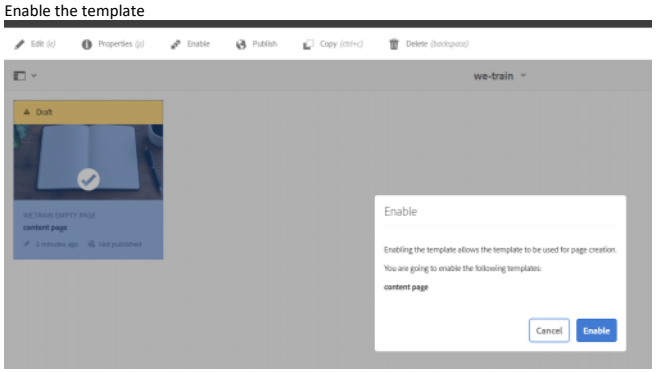
Now we have the 3 option in drag and drop
Create one more layout container
With same property and unlock the above container



Responsive grid is inner layout container
Root is out layout container

Policies inside templates is local property
Policies inside WCM is global property

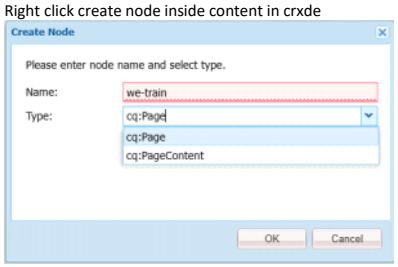
Task 4
Then
Enable the template



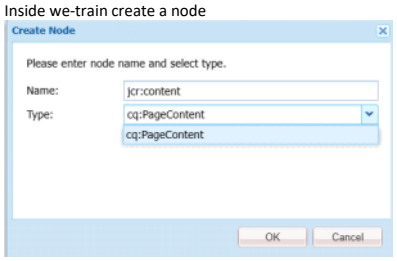
Creating Pages from Editable Templates

Exercise 2:Creating a page

Task 1:
Right click create node inside content in crxde

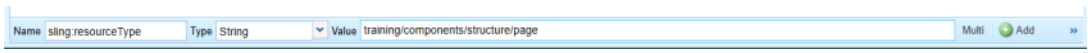


Inside we-train create a node

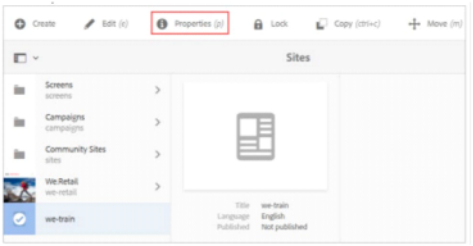


create a property for jcr:content



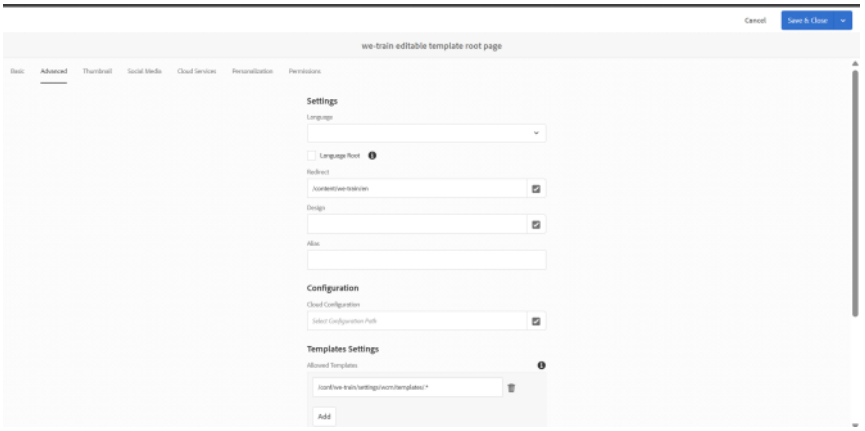


17. In your AEM author instance, click Adobe Experience Manager.
18. Click the Navigation icon on the left panel and click Sites. The Sites console opens.
19. Click the thumbnail beside the we-train page and click Properties as shown. The Properties wizard opens.



20. Ensure you are in the Basic tab and enter We.Train in the Title field.
21. On the Advanced tab:
    a. In the Redirect field, enter /content/we-train/en.
    b. Scroll down and click Add under Allowed Templates.
    c. Enter /conf/we-train/settings/wcm/templates/.* in the Allowed Templates field.
22. Click Save & Close. The form has been submitted successfully message appears.

To view the properties that you added to content root in JCR:



Task2:

Session-11 client libraries

In **Adobe Experience Manager (AEM)**, a Client-Side Library (configured via a cq:ClientLibraryFolder node) is used to organize, manage, and optimize CSS and JavaScript files

Categories:
The categories property acts as the **unique identifier or alias** for your client library.

- **Type:** String or String Array (Multi-valued)
- **Purpose:** Since you rarely reference a client library by its literal JCR repository path, you use the category name to include it in your HTML pages, HTL components, or page policies.
- **Behavior:** Because it is multi-valued, a single client library can belong to multiple categories.

Dependencies

The dependencies property defines a list of **other client library categories that must be loaded before** this library can execute.

- **Type:** String Array (Multi-valued)
- **Purpose:** Used for managing load order, typically when your custom script relies on external foundations or vendor frameworks (like jQuery, React, or Bootstrap).
- **Behavior:** When AEM generates the HTML links for your client library, it automatically detects dependencies and ensures they are included earlier in the page DOM. Dependencies are **transitive**; if library A depends on B, and B depends on C, loading A will safely load C and B first.

Embed

The embed property **physically merges the code** of specified target client libraries directly into this client library's final output file.

- **Type:** String Array (Multi-valued)
- **Purpose:** Primarily used for production optimization to **reduce HTTP requests** by bundling multiple smaller, component-specific libraries into a single master payload. It is also used to pull in utility code that shouldn't be exposed directly to the public.

AllowProxy

The allowProxy property is a security toggle that enables a client library stored securely under /apps to be **accessed publicly via a safe proxy servlet path**. [1, 2]

- **Type:** Boolean (true / false)
- **Purpose:** Security best practices isolate the /apps directory so anonymous end-users cannot look into the application structure. Setting allowProxy="{Boolean}true" instructs AEM to serve those assets safely via the /etc.clientlibs/ path instead.
- **Behavior:** If a clientlib is located at /apps/myproject/clientlibs/main and has allowProxy=true, the dispatcher and browser can safely access it via /etc.clientlibs/myproject/clientlibs/main.js.

Client-Side Libraries
standard way to include a client-side library (a JS or a CSS file) in the HTML of a page is to include a <script> or <link> tag in the Java Server Page (JSP) containing the path to the JS/CSS file. Although this approach works in AEM, it can lead to problems when pages and their constituent components become complex. In such cases, multiple copies of the same JS library may be included in the final HTML output.
To avoid this issue, AEM uses client-side library folders to organize the client-side libraries logically.
The basic goals of client-side libraries are to:
• Store CSS/JS in small discrete files for easier development and maintenance
• Manage dependencies on third-party frameworks in an organized fashion
• Minimize the number of client-side requests by concatenating CSS/JS into one or two requests
• Minimize CSS/JS that is delivered to optimize speed/performance of a site

-categories
-embed
-dependencies
-css.txt
-js.txt
-allowproxy

Folder will have primary type of cq:cliendlibraryfolder

7	jcr:primaryType	Name	cq:ClientLibraryFolder	true

For a node to become a client library it should fulfill below things
1. Its primary type should be cq:ClientLibraryFolder.
2. It should have a property "categories". If a folder doesn't have this property it cannot be client library.
3. It either should have a css.txt file or a js.txt file. Without any of these files this folder cannot be a client library. (list of all CSS and js files that should be compiled into this library)

How to create a client library

Exercise 1: Add Content to a page

In this exercise, you will add some content to a page. After completing this exercise, you will see the content without a design. In later exercises, you will apply the We.Train design and you will see the content transform to the We.Train design standards.

1. In your AEM author instance, navigate to Sites > We.Train > English page.
2. Click the English page thumbnail, and click Edit (e) on the header bar to open the English page.
3. In the left panel, click the Components icon. The available components are displayed.
4. Drag and drop the Content Fragment component onto the layout container.
5. In the left panel, click the Assets icon. The available assets are displayed.
6. Filter the assets based on Content Fragments by clicking the drop-down menu and selecting Content Fragments, as shown:



7. Drag and drop the Arctic Surfing in Lofoten fragment onto the Content Fragment component. Notice how the content has no design currently.

Exercise 2: Create a client-side library

In this exercise, you will create a base client-side library. This base library will help compile the site's css/js into a single client library.

1. Navigate to Adobe Experience Manager > Tools > CRXDE Lite. The CRXDE Lite page opens.
2. Navigate to the /apps/training folder.
3. Right-click the training folder and click Create > Create Folder. The Create Folder dialog box opens.
4. Type clientlibs in the Name field and click OK. The folder is created under the training folder.
5. Click Save All.
6. Right-click the clientlibs folder and click Create > Create Node. The Create Node dialog box opens.
7. Update the following fields:
   a. Name: clientlib-base
   b. Type: cq:ClientLibraryFolder
8. Click OK. The folder is created.
9. Click Save All from the actions bar.
10. Select the **clientlib-base** folder, and add the following properties:

Name	Type	Value
categories	String[]	we.train.base
embed	String[]	we.train.site we.train.font-awesome
allowProxy	Boolean	true

Note: To add the below properties, you must add the property, click Add, and then click Save All. Follow the same procedure for other properties.

Note: To enter a String Array (String[]), select String from Type field, and then click the Multi button. Remember, the Multi button will be selected until you deselect it. If a dialog box appears with the string array information, click OK.

11. When updating the embed property, the Edit embed dialog should look, as shown:
12. Click Save All from the actions bar.
13. Right-click the **clientlib-base** node and click **Create > Create File**. The **Create File** dialog box opens.
14. Enter **css.txt** in the Name field and click OK. The file is created under the clientlibs folder.
15. Click **Save All** from the actions bar.
16. Perform steps 13 through 15 to create the **js.txt** file.

Next, you will upload the **we-train-design-package** to the JCR. This package contains the css/js that you will use in the We.Train site structure.

To install the we-train-design-package:

17. From CRXDE Lite, click the Package icon from the header bar, as shown. The Package Manager page opens.



18. Click Upload Package from the actions bar. The Upload Package dialog box opens.
19. Click Browse.
20. Navigate to the Creating Client-side Libraries in Adobe Experience Manager exercise folder provided to you and double-click the **we-train-design-package.zip** package.
21. Click OK in the Upload Package dialog box. The package is uploaded.
22. In the **we-train-design-package.zip** section, click Install, as shown. The Install Package dialog box opens. Notice the filters on the package. This dialog box shows the files and folders that will be installed in your **/apps/training/clientlibs** folder on JCR.



23. Ignore the Advanced Settings and click Install. The package is installed on your instance.
24. Click the Develop icon from the header bar, as shown. The CRXDE Lite page is refreshed.



25. Navigate to the /apps/training/clientlibs folder and observe the client libraries that are installe the package.



Exercise 3: Add a client-side library to a page component

In this exercise, you will include the client libraries directly in the page component with convenience scripts called customheaderlibs.html and customfooterlibs.html. These scripts enable a developer to hard code a mandatory design to the page component.

1. Click http://localhost:4502/crx/de/index.jsp if you are not already in CRXDE Lite. The CRXDE Lite page opens.
2. Navigate to the **/apps/training/clientlibs/clientlib-base** node. Observe the property **categories=we.train.base.** You will use this category name when adding the clientlib to a page component.

To add the HTML files:

3. In CRXDE Lite, navigate to the /apps/core/wcm/components/page/v2/page component.

4. Right-click the customheaderlibs.html file and click Copy.
5. Navigate to the /apps/training/components/structure/page.
6. Right-click the page node and click Paste to paste the file you copied in step 4.
7. Similarly, copy the customfooterlibs.html file from /apps/core/wcm/components/page/v2/ page folder and paste it to the /apps/training/components/structure/page node.
8. Click Save All.

To add code to the .html files:

9. Navigate to the exercise files, open customheaderlibs.html in any text-pad or notepad, copy the script, and paste it into customheaderlibs.html in CRXDE Lite, as shown:



10. Click save all.
11. Repeat step 9 but copy-paste the script from customfooterlibs.html to CRXDE Lite.
12. Click Save All.

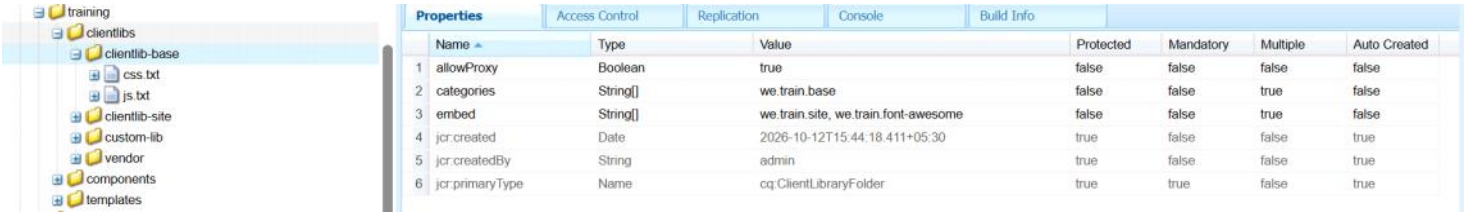
You can verify if the CSS and JavaScript client libraries were added to the top and bottom of the page from the HTML source of a page.

13. Click CRXDE Lite from the header bar. You will be taken to the AEM Navigation page.
14. Click Sites > We.Train site.
15. Click the thumbnail on the English page, and then click Edit (e) from the actions bar. The English page opens in a new tab.

On the English page, you should see that a different design is applied to the content fragment you added earlier.



Explanation

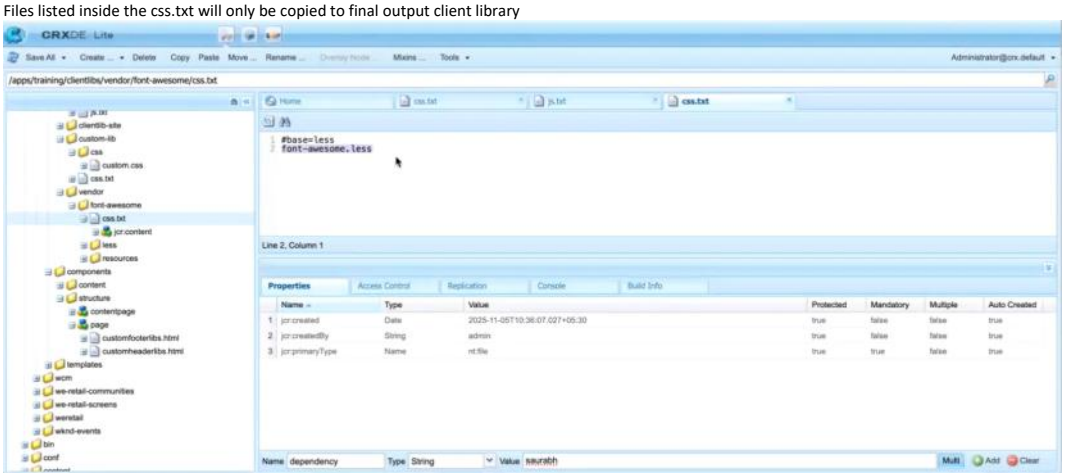


Here categories property defines the name of client library(it can have multiple name that's why it is a multivalued property)
Here it combines 5 files

If same we.train.base is available in other it will include its own file and also here the 5 files and combine it into one with 7files

Client libraries can have same name (in that case both will be merged)

Files listed inside the css.txt will only be copied to final output client library



Using HTL

In HTL, client libraries are loaded through a helper template provided by AEM, which can be accessed through `data-sly-use`. Three templates are available in this file, which can be called through `data-sly-call`:

- **css** - Loads only the CSS files of the referenced client libraries.
- **js** - Loads only the JavaScript files of the referenced client libraries.
- **all** - Loads all the files of the referenced client libraries (both CSS and JavaScript).

Each helper template expects a `categories` option for referencing the desired client libraries. That option can be either an array of string values, or a string containing a comma separated values list.

8. Open the file and type the following text to identify the root of the path of the source files:

   ```
   #base=*[root]*
   ```

   Replace * `[root]`* with the path to the folder that contains the source files, relative to the TXT file. For example, use the following text when the source files are in the same folder as the TXT file:

   ```
   #base=.
   ```

   The following code sets the root as the folder named mobile below the `cq:ClientLibraryFolder` node:

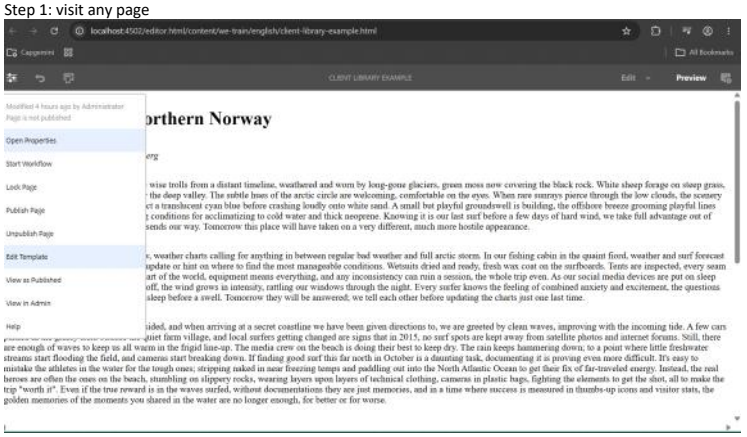
   ```
   #base=mobile
   ```

9. On the lines below `#base=[root]`, type the paths of the source files relative to the root. Place each file name on a separate line.

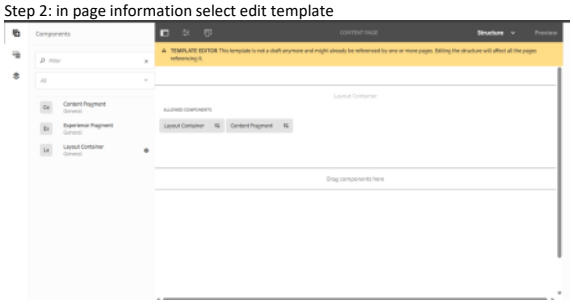
10. Click **Save All**.

Adding a client library to Template(this is via configuration)

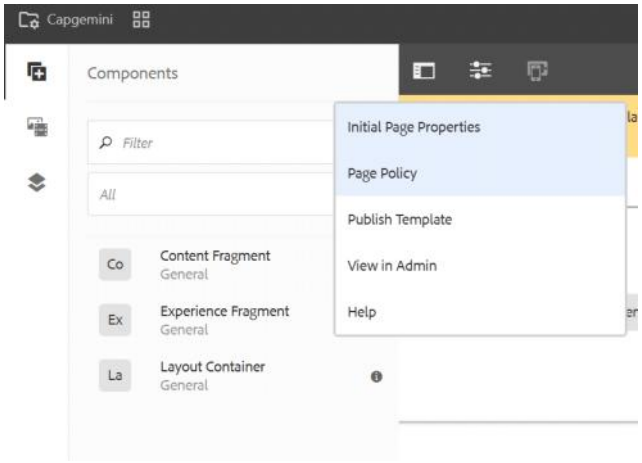
Step 1: visit any page



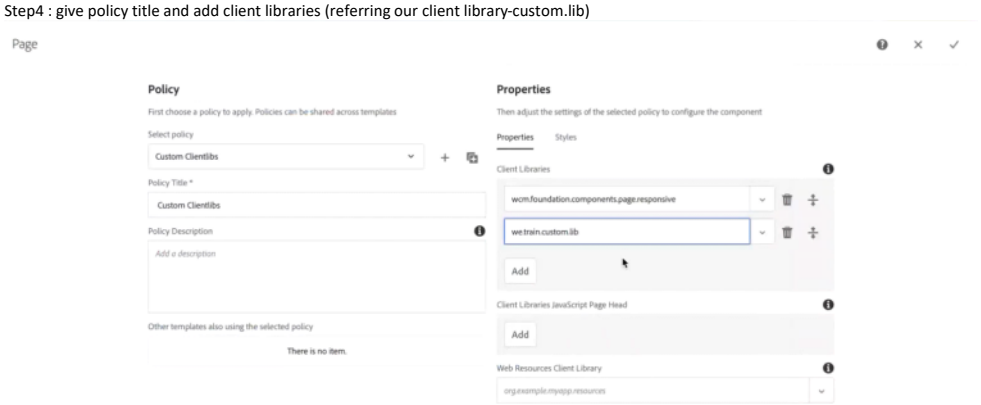
Step 2: in page information select edit template



Step 3: select the page policy in page information



Step4 : give policy title and add client libraries (referring our client library-custom.lib)

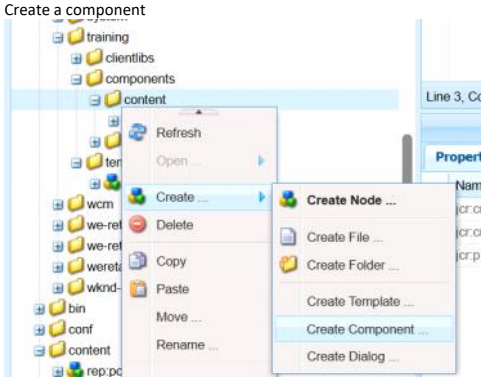


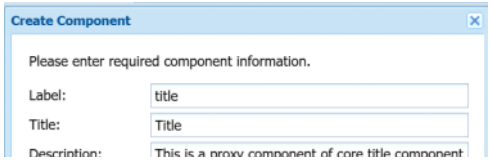
Note: The Content Page template is already enabled, which means the changes you make affect all pages that were built from the template.

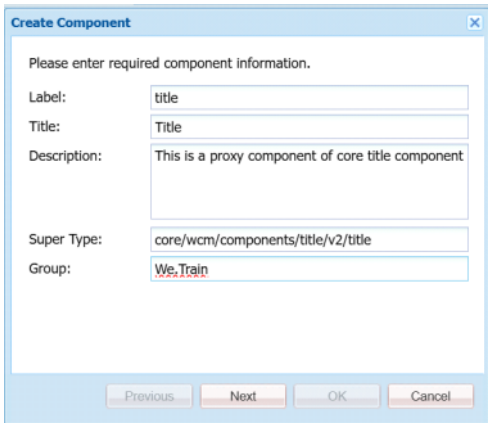
Session 12 - Core components
Adobe Experience Manager (AEM) provides a wide range of component implementations on a standard instance by default
Proxy components define the required component name and group to display to page authors, and refers to a Core component as their super type
Core components must not be directly referenced from the content. To avoid direct reference, the Core components are added to a hidden component group (.core-wcm or .core-wcm-form). This prevents displaying Core components in the Template Editor.

Hands-on
Task 1 : Create proxy components

In CRXDE Lite, navigate to the /apps/training/components/content folder.
Create a component







Next we need to do the same for all core components

/content/breadcrumb*
/content/carousel^*
/content/image^*
/content/list
/content/separator
/content/sharing
/content/tabs^*
/content/teaser
/content/text*
/form/button
/form/container*
/form/hidden
/form/options
/form/text
/structure/search*
/structure/languagenavigation

Instead we import the package given by Saurabh (exercise session 12)

Note: The Caret (^) symbol next to some of the components indicates that the components have additional node structures that are created with the content package.
Note: The Asterisk (*) next to some of the components indicates that the components have an accompanying client library that must be added. You will add these in a later exercise.

Task 2: Add core component client library

Exercise 2: Add core component client libraries

As you are using multiple core components as proxy components, you must include the required CSS from the core components. You will add these to the clientlib-base client library with the core component category names.

1. In CRXDe Lite, navigate to /apps/training/clientlibs/clientlib-base.
2. Double-click the embed property. You should see the clientlibs you added in a previous task.
3. To add more category names, click the plus (+) icon, as shown, and add the **core.wcm.components.breadcrumb.v2** category name.

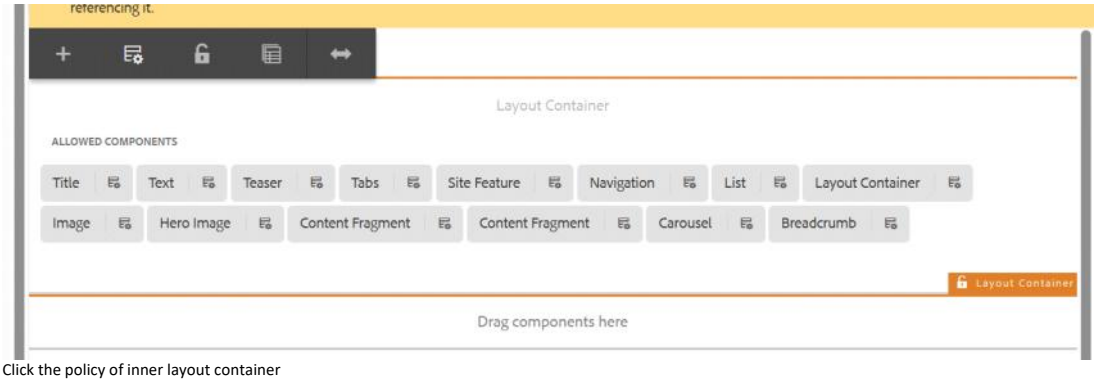


4. Repeat step 3 to add the following category names as separate entries:

core.wcm.components.carousel.v1
core.wcm.components.image.v2
core.wcm.components.tabs.v1
core.wcm.components.search.v1
core.wcm.components.form.text.v2
core.wcm.components.form.container.v1

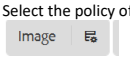
Task 3:

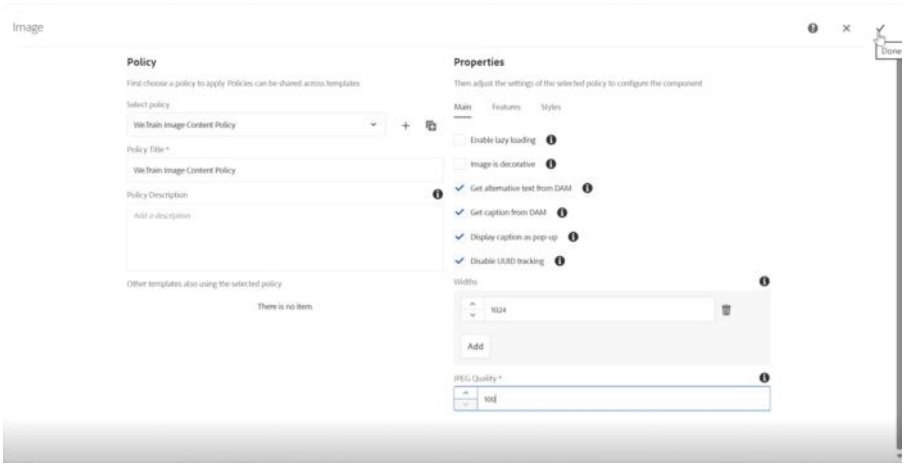
Navigate to template editor



Click the policy of inner layout container
And enable the new group we created we.train
Which contain all the core components listed above

Task 4:
Select the policy of image





This demonstrates that we can even edit policy for each individual components

Session -13 components Hands-on

Structure of a Component

The component structure includes:

- Root node
- Vital properties
- Vital child nodes

Root Node

It is the hierarchy node of the component and represented as node `<mycomponent>` (type `cq:Component`).

Vital Properties

The vital properties of a component are:

- jcr:title: Is the component title. For example, this property is used as a label when the component is listed in the components browser.
- jcr:description: Is the description for the component. You can use this property as a mouse-over hint in the components browser.
- cq:icon: Is the string property pointing to a standard icon in the Coral UI library to display in the component browser.

Vital Child Nodes

The vital child nodes of a component are:

- cq:editConfig (cq:EditConfig): Defines the edit properties of the component and enables the component to appear in the components browser
- cq:childEditConfig (cq:EditConfig): Controls the author UI aspects for child components that do not define their own cq:editConfig
- cq:dialog (nt:unstructured): Is the dialog for this component and defines the interface allowing the user to configure the component and/or edit content.
- cq:design_dialog (nt:unstructured): Helps edit the design of a component

Helper classes in AEM HTML file which will call Java class or these server side helper class and this helper class will have the business logic and will return output into HTML file.

MVC Design pattern -:

M -> Model
V -> View
C -> Controller

In a website View -> HTML which gets displayed on the browser which is static in nature mostly.
Model -> Now to provide some dynamic data to your HTML you need your helper classes or backend code which will do the business logic and send you the dyanmic content to be displayed on your view(HTML)
Controller -> Which can connect View with Model

In AEM to write business logic we use helper classes, there are two type of such helper classes we have
1. Server side JS -> Deprecated but still in use.
2. Sling Model -> This is what is our focus and this is only we will learn in detail.

Exercise 1: Create a header component (without use of helper classes)

Synthetic resource:is a lightweight, in-memory representation of a resource that does not exist as an actual node in the JCR

In Adobe Experience Manager (AEM), **data-sly-resource** is an HTML Template Language (HTL / Sightly) block statement used to include the output of rendering a specified resource through the Apache Sling resolution and rendering process

3. Navigate to the /apps/training/components/structure folder.
4. Right-click the structure folder and click **Create** > **Create Component**. The Create Component dialog box opens.
5. Enter the following details in the dialog box:
   a. **Label: header**
   b. **Title: We.Train Header**
   c. **Group: We.Train.Structure**
6. Click Next. The Advanced Component Settings are displayed. Leave these settings as is
7. Click OK. The component is created.
8. Click Save All from the actions bar.

11. Right-click header.jsp, select Rename, and change the file name to header.html. Click Save All from the actions bar.
12. Click Save All from the actions bar.
13. Double-click the header.html to open it in the editor on the right for editing.
14. Navigate to the **basic_header.html** file located in the exercise folder provided to you, copy the code, and paste the code in **header.html.**
15. Click **Save All** to save changes.

17. Right-click the header component and click Create > Create Node. The Create Node dialog box opens.
18. Provide the following details:
    a. Name: cq:dialog

---

b. Type: nt:unstructured

19. Click OK. The node is created.
20. Click Save All.

To add the header component to the template:
22. Click Tools > Templates. The Templates console opens.
23. Click the We.Train folder, and select the Content Page template by clicking the Select icon, as shown:



24. Click Edit (e) from the actions bar. The Template Editor opens.
25. Select the Layout Container [Root] and click the Policy icon from the component toolbar, as shown. The Layout Container policy wizard opens.



To add the header component to the template:

22. Click Tools > Templates. The Templates console opens.
23. Click the We.Train folder, and select the Content Page template by clicking the Select icon, as shown:



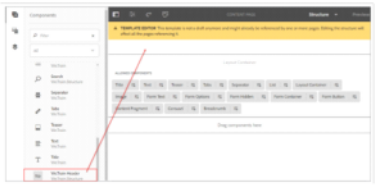
24. Click Edit (e) from the actions bar. The Template Editor opens.
25. Select the Layout Container [Root] and click the Policy icon from the component toolbar, as shown. The Layout Container policy wizard opens.



26. In the Properties section, on the Allowed Components tab, scroll down the components list, and select the We.Train.Structure checkbox. This group contains the header component.
27. Click Done (checkmark in the upper right) to save the changes.

We.Train.Structure policy will automatically have access to the structure components (header) that you have built.

28. Click the Toggle Side Panel icon, if the left panel is not visible, and click the Components icon. The Components browser opens.
29. Drag the We.Train Header component from the panel and add it above the Layout Container, as shown. The Header Component is added to the template.



After adding the header component, you must set policies for the two proxy components.

30. Select the Language Navigation component and click the Policy icon from the component toolbar.
31. In the Policy section, type **We.Train LangNav** Content Policy in the Policy Title field.
32. In the **Properties** section, type **/content/we-train** in the Navigation Root field.
33. Click Done (checkmark) to save the changes.
34. Select the Search component and click the Policy icon.

Note: You could have configured the language navigation and search components by using a cq:template node too—similar to the carousel and tab proxy components. For example, review /apps/weretail/components/structure/header in CRXDE Lite.

35. Click http://localhost:4502/editor.html/content/we-train/en.html. The English page of the We.Train site with the header component (and its child pages) opens, as shown:

You created the component, added it to the template, and verified it on the page. Now, you can begin a deeper development with the header.html script.

36. Click http://localhost:4502/crx/de. The CRXDE Lite page opens.
37. Navigate to the **/apps/training/components/structure/header** node.
38. Double-click the **header.html** to open the script for editing.

39. Update the header.html script by replacing the existing code with new code from **basic_full_header.html** exercise file provided to you.
40. Click Save All.
41. Review the code you added. Notice in the following code snippet of header.html, there is an account navigation bar and a responsive site navigation bar. Most anchors are href='#' that help mock the page for the business logic. Notice the **${header.rootPage.path}** and **${!header.anonymous}** variables from the business logic. As the header variable is not defined, it is not evaluated.
42. Navigate to the English page of the **We.Train** site and refresh the page.
43. Click Preview, and in the Preview mode, click the Emulator icon.
44. Click Desktop, iPhone 8 Plus, and iPhone X at the top, and notice that the header is responsive.

iPhone 8 Plus

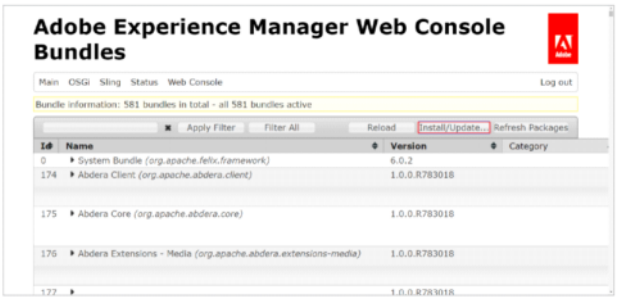
Exercise 2: Add JavaScript business logic to the header component

Exercise 3:Add a Sling Model business logic to the header component
Sling module uses used compiled java class , bcz AEM cannot run java without JDK
Only compiled classes can we available in server

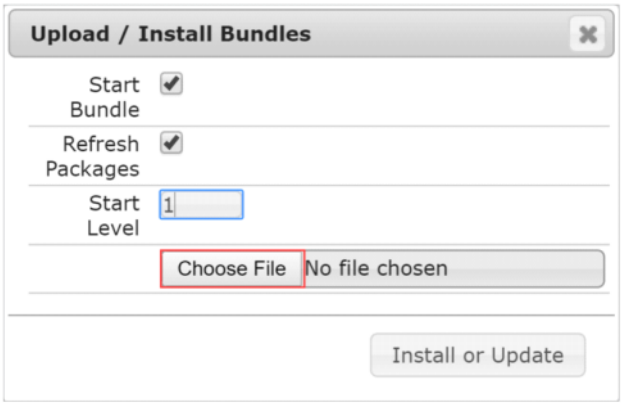
Go to Adobe Experience Manager->> Operations > Web Console

Click OSGi from the header bar, and click Bundles from the drop-down menu, as shown. The Adobe Experience Manager Web Console Bundles page opens.

Click **Install/Update** from the actions bar, as shown. The **Upload/Install** Bundles dialog box opens.



Select the **Start Bundle** and Refresh Packages checkboxes, enter 1 in the **Start Level** field, and click **Choose File**, as shown. The **Open** dialog box appears.



Navigate to the exercise folder on your file system and double-click the we.train.core-0.0.1- SNAPSHOT.jar file. The file is selected. 7. Click Install or Update in the Upload / Install Bundles dialog box. The Adobe Experience Manager Web Console Bundles page refreshes when the installation is complete.]

To verify

To verify if the bundle is installed successfully on your instance:

8. Click **Status** on the header bar and select **Sling Models** from the drop-down menu. The Adobe Experience Manager Web Console Sling Models page opens.
9. Press Ctrl + F (Windows) or Command + F (Mac) on your keyboard and type **training/components/structure/header** in the search field. Notice that the com.adobe.ats.core.models.Header - training/components/structure/header is under **Sling Models Bound to Resource Types** *For Requests (if you cannot find this, use Ctrl+F or Command F). This confirms the bundle is installed successfully.

Now change the header.html code to model header.html code in exercise

Exercise 4: Create an edit dialog for the component

Exercise 5: Add an editconfig node to the component

Exercise 6: Add a component client library to the component

Exercise 7: Create a design dialog for a component

Difference between cq:dialog and cq:design_dialog

Feature	`cq:dialog` (Regular Dialog)	`cq:design_dialog` (Design Dialog)
Purpose	Configures content and functional aspects of a specific component instance (e.g., text, image path, links).	Configures visual styling, layout, and global options that apply across the template or site (e.g., allowed components, default styles).
Authoring Mode	Accessible in Edit Mode of a page.	Accessible in Design Mode or Template/Policy configuration.
Data Storage Location	Stored at the page level under `/content/<page>/jcr:content/<component-node>`.	Stored at the template/design level under `/etc/design/...` (or `/conf/...` in modern AEM).
Scope	Affects only the single instance of the component dropped on that specific page.	Affects all instances of the component that share that design or template configuration.
Node Name	`cq:dialog` (Touch UI) / `dialog` (Classic UI)	`cq:design_dialog` (Touch UI) / `design_dialog` (Classic UI)
Requirement	Needed by almost all content-driven components.	Optional; only used when a component needs template-wide design controls (e.g., `Title`, `Image`, or `Parsys`).

JDK(Java Development Kit)

- which includes all the tools, executables, and binaries required to compile, debug, and execute a Java Program
- JDK is platform dependent i.e. there are separate installers for Windows, Mac, and Unix systems.
- JDK includes both JVM and JRE and is entirely responsible for code execution

JRE (Java Runtime Environment)

- which is the implementation of JVM i.e. the specifications that are defined in JVM are implemented and create a corresponding environment for the execution of code.

Day - 12
Session -14 custom component

Creating a placeholder

```
<sly data-sly-use.model="my.site.my_site.core.models.FirstModel">

<div data-sly-test.empty="${wcmmode.edit  && !model.firstNumber && !model.secondNumber}" class="cq-placeholder"
        data-emptytext="${component.title}">
</div>
```

or

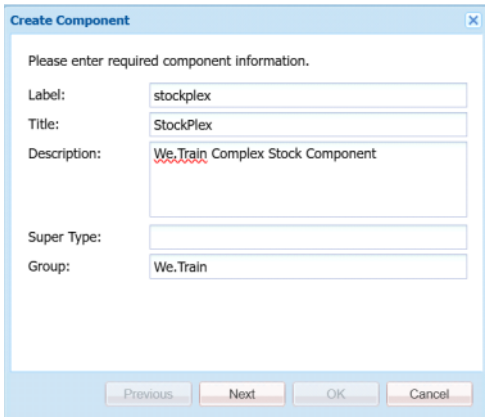
```
<h1 data-sly-use.title="com.adobe.cq.wcm.core.components.models.Title"
    data-sly-use.template="core/wcm/components/commons/v1/templates.html"
    data-sly-element="${title.type}"
    data-sly-test.text="${title.text}">${title.text}</h1>

    <!--/* Subtitle added to HTL */-->
<h2 data-sly-test="${properties.subtitle}">${properties.subtitle}</h2>

<sly data-sly-call="${template.placeholder @ isEmpty=!text}"></sly>
```

Session 14
Exercise 1: create a custom component

Navigate to the /apps/training/components/content ------> create component



Right-click the stockplex component and click **Create** > **Create Node**. The Create Node dialog box opens.

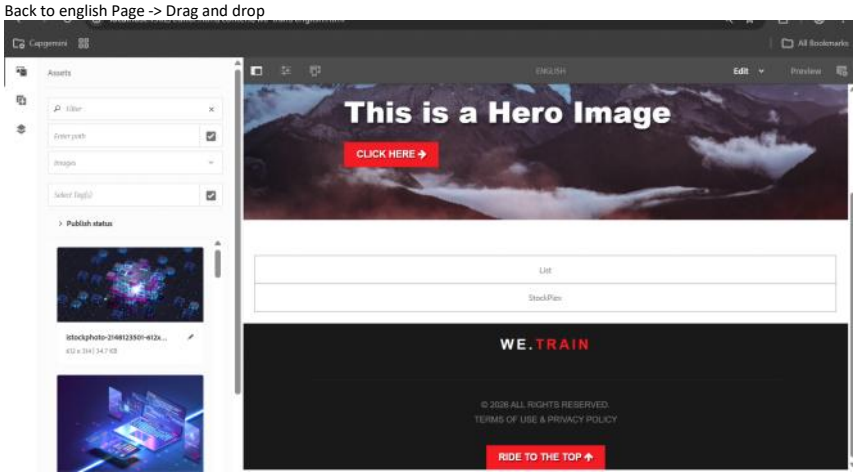
Update the following details:

a. Name: cq:dialog

b. Type: nt:unstructured

Go to template in layout container enable the stockplex component in policy

Back to english Page -> Drag and drop



Exercise 2: add dialog field validation to custom component

Update the stockplex.html script by replacing the existing code with new code from **dialog_stockplex.html** located in the exercise folder
Select the cq:dialog node that is below the stockplex component.

Right-click the cq:dialog node and click Delete. The node is deleted.

/apps/core/wcm/components/title/v2/title/cq:dialog node, right-click the cq:dialog node, and click Copy.

Navigate to the /apps/training/components/content folder, right-click the stockplex node, and click Paste.

On the cq:dialog node, update the following properties in the Properties section:

a. Change the value of jcr:title to Stockplex.
b. Right-click the helpPath property and click Delete to delete it.

Navigate to the /apps/training/components/content/stockplex/cq:dialog/content/items/tabs/items/properties/items/columns/items/column/items node.

-Right-click the types node and click Delete to delete it
-Right-click the defaulttypes node and click Delete to delete it.

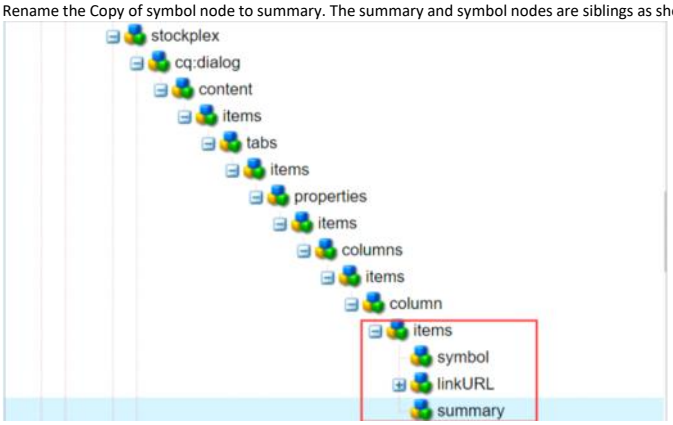
Navigate to the /apps/training/components/content/stockplex/cq:dialog/content/items/tabs/items/properties/items/columns/items/column/items/title node

Right-click the title node, click Rename, and rename it as symbol.

Name	Type	Value
name	String	./symbol
fieldLabel	String	Stock Symbol
fieldDescription	String	Enter a 4-character stock symbol

Right-click the symbol node, and click Copy

Right-click the items node that is above the symbol node, and click Paste.
Rename the Copy of symbol node to summary. The summary and symbol nodes are siblings as shown:



Name	Type	Value
name	String	./summary
fieldLabel	String	Summary of Stock
fieldDescription	String	Enter a summary description of the stock

Right-click the items node that is above the symbol node, and click Paste to paste the node.
Rename the Copy of symbol node to stockdetails

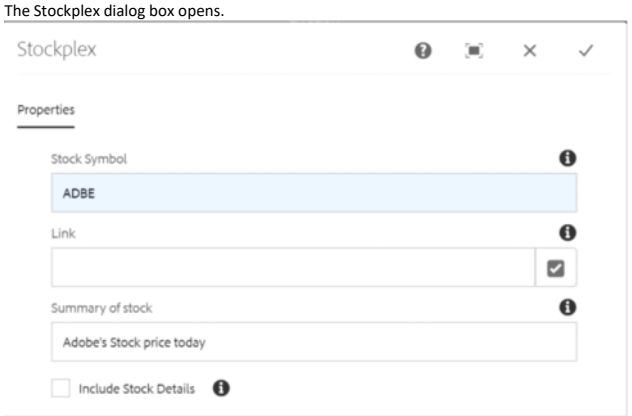
Name	Type	Value
name	String	./showStockDetails
fieldDescription	String	Include requestDate, upDown, openPrice, range high/low, volume, and others
sling:resourceType	String	granite/ui/components/coral/foundation/form/checkbox

Add the following new properties to the stockdetails node:

Name	Type	Value
text	String	Include Stock Details
value	Boolean	true

Select the fieldLabel property, right-click it, and click Delete from the list.

Navigate to the tab where the English page is open
Select the stockplex component you added in a previous task, and click the Configure (wrench) icon.
The Stockplex dialog box opens.



Click done

Now for validataion

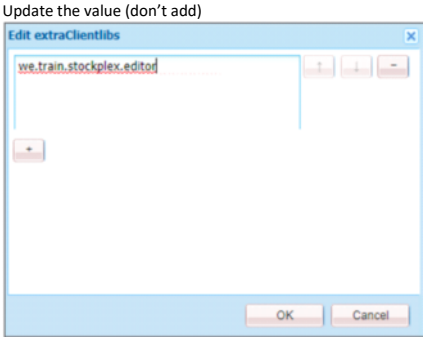
Open package manager and upload stockplex-clientlibs-all.zip package from the exercise folder

Install the package.

Navigate to the /apps/training/components/content/stockplex/cq:dialog node

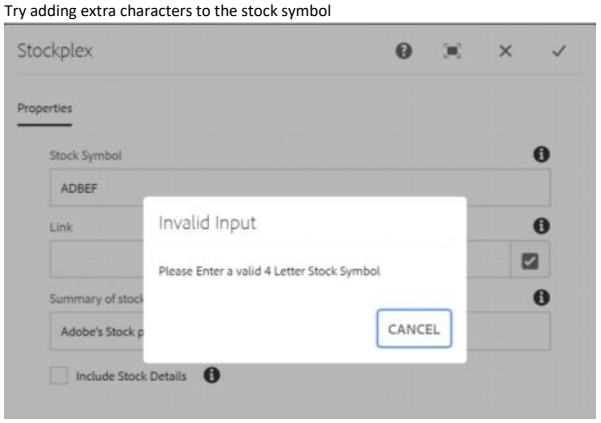
In Properties section, double-click in the extraClientlibs. The Edit extraClientlibs dialog box opens.

Update the value (don't add)



Now go back to English page
Select the stockplex component, and click the Configure (wrench) icon. The Stockplex dialog box opens.

Try adding extra characters to the stock symbol



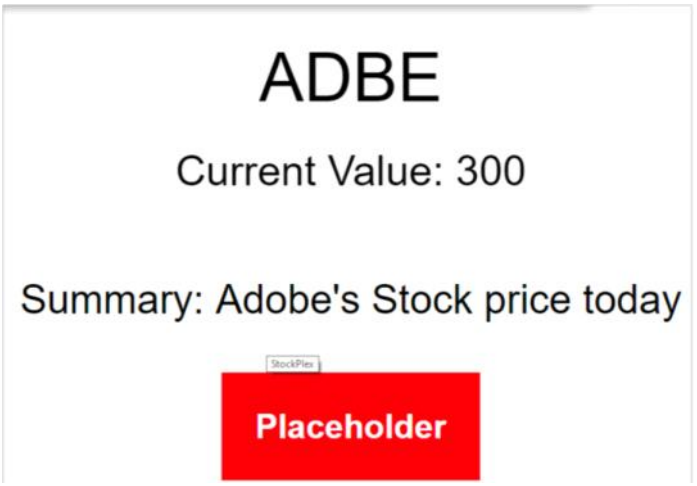
Exercise 3: add style to custom component(via authoring)

Navigate to the /apps/training/components/content folder.

Select the stockplex node and double-click the stockplex.html to open the script for editing.

Update the stockplex.html script by replacing the existing code with the new code in **clientlibs_stockplex.html** located in the exercise folder.

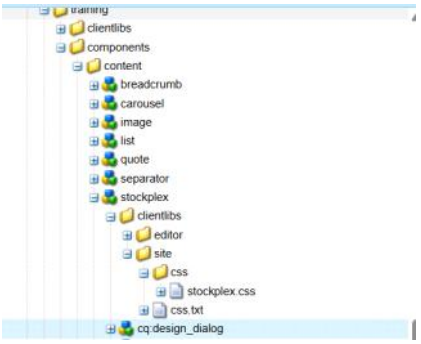
Click **Save All**.



Exercise 4: Add a style system to the custom component

Navigate to the /apps/core/wcm/components/text/v2/text node, right-click the cq:design_dialog node, and click Copy to copy the node.

Navigate to the /apps/training/components/content/stockplex node, and click Paste to paste the node.



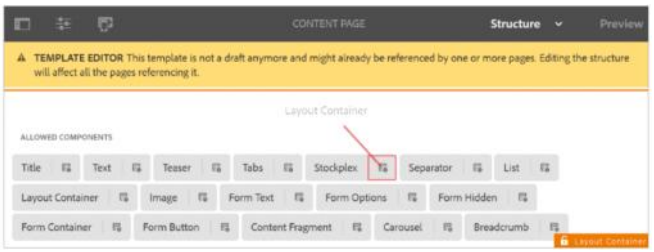
Navigate to the /apps/training/components/content/stockplex/cq:design_dialog/content/ items/tabs/items node, right-click the plugins node, and click Delete

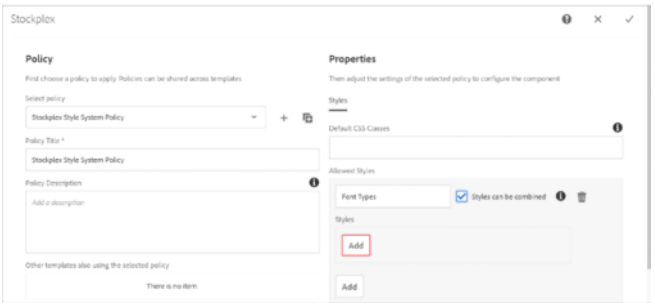
Click the **Tools** (hammer) icon. The Tools console opens.

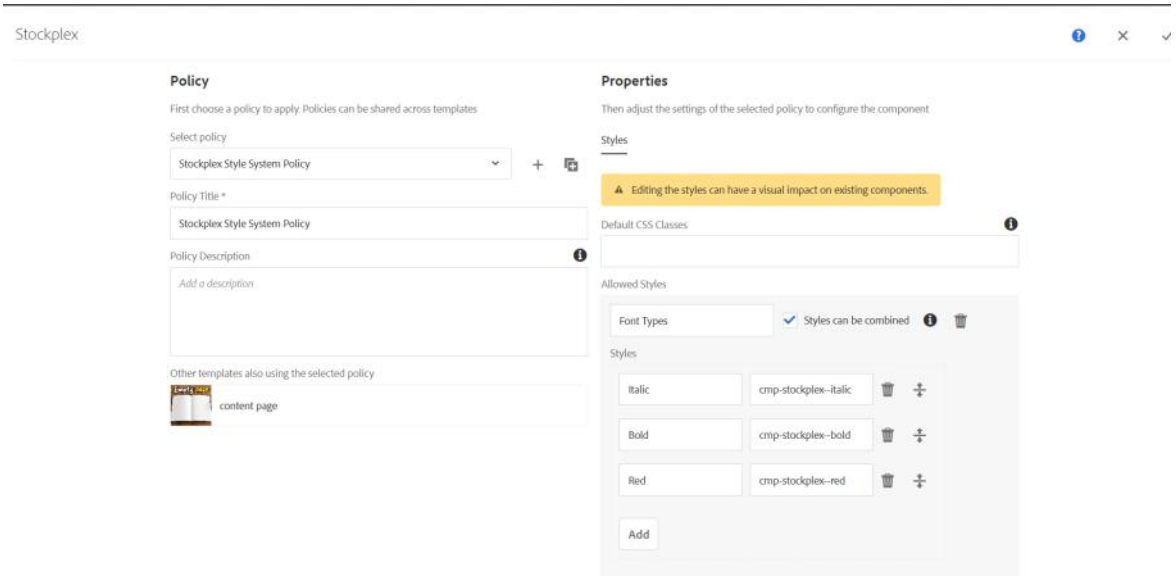
Ensure you are in the **General** section and click the Templates tile. The Templates console opens.

Click the **We.Train** folder, select the **Content Page** template, and click Edit (e) from the actions bar. The **Content Page** template editor opens on a new tab.

15. In the **Layout Container** component, click the **Policy** icon next to the **Stockplex** component, as shown. The **Stockplex** policy wizard opens.



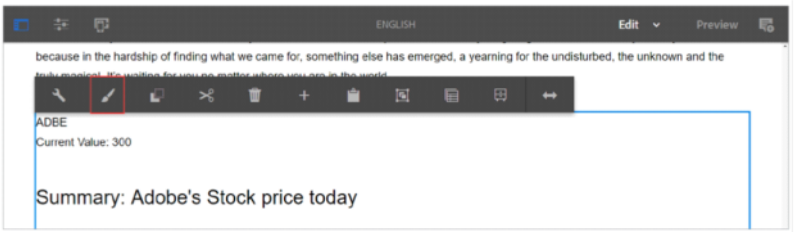


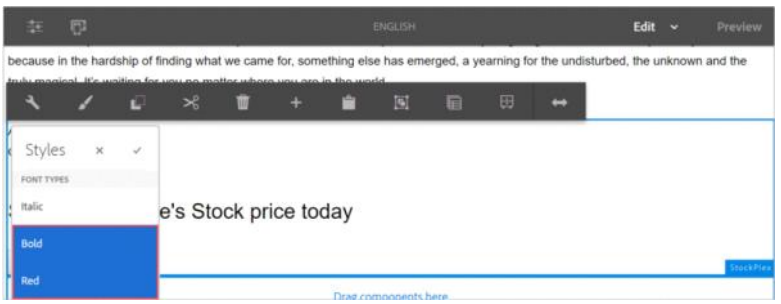


Italic - cmp-stockplex--italic
Bold - cmp-stockplex--bold
Red - cmp-stockplex--red

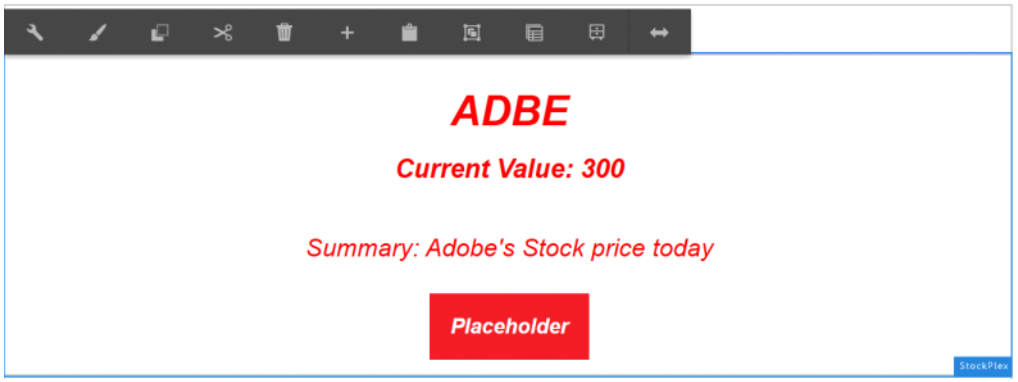
Navigate to the English page of the We.Train site, and refresh it.

Select the **Stockplex** component. You should now see the **Styles** (paintbrush) icon in the component toolbar, as shown:







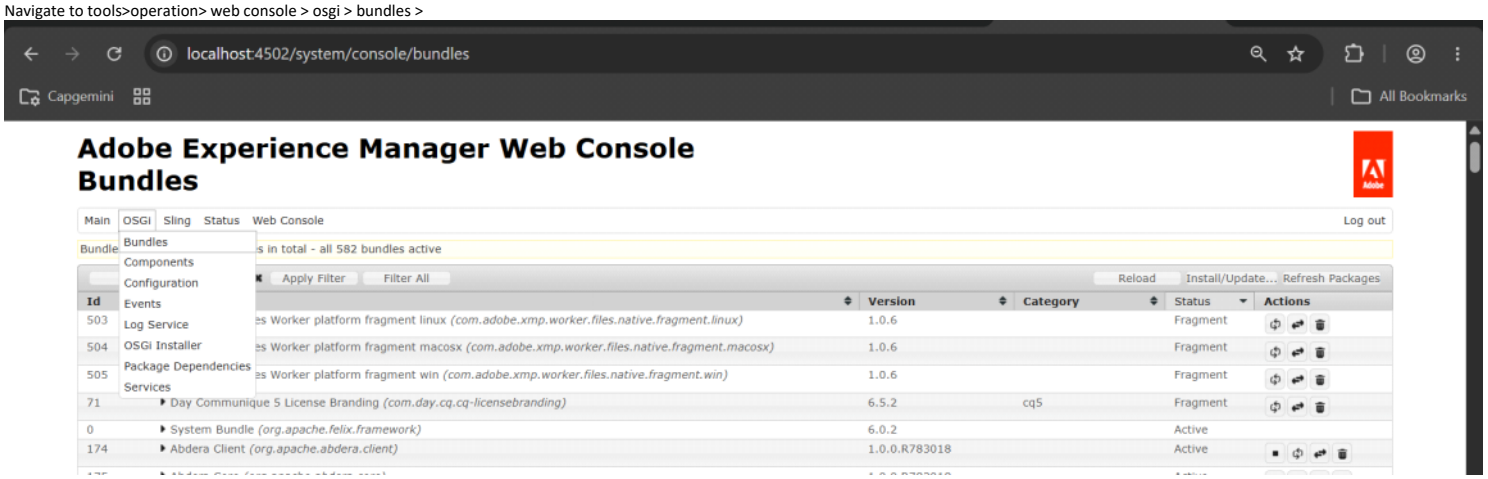


Exercise 5: Add a sling model as a the business logic

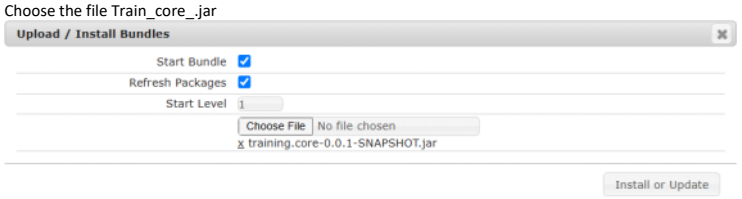
The Sling Model you will install is called Stockplex.java and is compiled into an OSGi bundle. Observe the raw java code in your exercise files.
To do that install training.core-0.0.1-SNAPSHOT.jar bundle(provided in exercise folder) from felix console.

Hands-on

Navigate to tools>operation> web console > osgi > bundles >



Choose the file Train_core_.jar



Then Update the stockplex.html code to slingmodel_stockplex.html from the exercise files provided to you

Then refresh the page Englishg



Install Maven 3.6.3 version

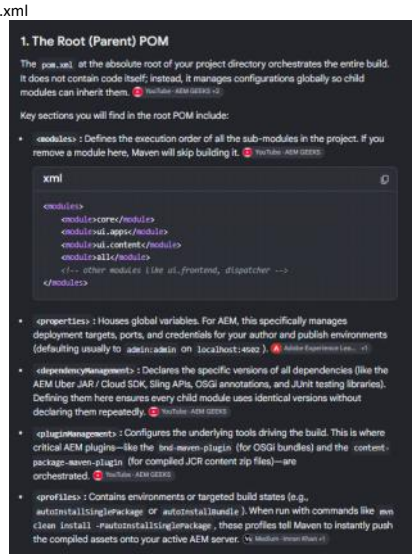
Maven compiles all the java files and creates/build .class file and combines all then into one bundle and give to server

Day-13 16th oct
Session 15

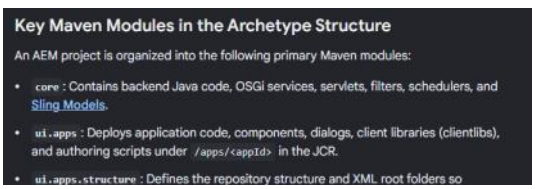
Maven:an open-source build automation and project management tool primarily used for Java applications
pom.xml stands for Project Object Model, and it is the core XML configuration file located at the root directory of every Maven project. It acts as the single source of truth and the blueprint for the build system, telling Maven exactly what your project is, how to build it, and what external libraries it requires.

Key Functions of pom.xml	
Dependency Management	
Build Lifecycle Configuration	
Project Metadata	
Plugin Management	

Pom.xml



Structure of mvn archetype



Key Maven Modules in the Archetype Structure

An AEM project is organized into the following primary Maven modules:

- `core` : Contains backend Java code, OSGi services, servlets, filters, schedulers, and Sling Models.
- `ui.apps` : Deploys application code, components, dialogs, client libraries (clientlibs), and authoring scripts under `/apps/<appId>` in the JCR.
- `ui.apps.structure` : Defines the repository structure and XML root folders so subpackages install safely without circular dependency issues.
- `ui.config` : Deploys OSGi configurations (repository-based run-mode configurations) under `/apps/<appId>/osgiconfig` .
- `ui.content` : Deploys mutable content, initial site pages, experience/content fragments, and Context-Aware Configurations under `/content` , `/conf` , and `/ite-root` .
- `ui.frontend` : Manages front-end source files (CSS, SASS, JS, TypeScript, Webpack) or SPA frameworks (React/Angular).
- `dispatcher` : Holds the Apache HTTP Server and Dispatcher configuration files for caching and security (used for local testing and cloud deployments).
- `all` : A container package that bundles the `core` , `ui.apps` , `ui.config` , and `ui.content` subpackages into a single deployable package for AEM Author and Publish.
- `it.tests` / `ui.tests` : Contains integration tests and automated UI tests for the application. GitHub +5

All - contains compiled java class

Artifact means -> Final output of something

Artifactory is a place where you keep your artifacts

Maven keeps a centralized artifactory which is also called **Maven Central Repository** from where you can download dependencies for your project and also you can deploy your artifacts on maven central public repository. This Maven central public repository is public.

What is the artifact from core module -> It is jar file which consists compiled java classes and it is also called bundle.

Jar file(generic java) ~ bundle(specific to AEM)
For a jar to become a bundle it has to have some additional things with it then only we can call a jar as bundle.
That additional information is called meta information.
Like -> name, version, dependency, …….

Target foler is the folder in which maven generates aratifact for you after build

If AEM do not start
This location you go and delete whatever is below "repoinit" node and then restart your server. /crx-quickstart/launchpad/config/org/apache/sling/jcr/repoinit

Session 15
Sling Model
is an annotation-driven Plain Old Java Object (POJO) used to map repository content (JCR nodes), HTTP requests, and OSGi services into Java objects

Key Annotations

Annotation	Purpose
`@Model`	Declares the Java class as a Sling Model and defines adaptables.
`@ValueMapValue`	Injects component properties from the resource's value map.
`@OSGiService`	Injects available OSGi services into the model.
`@PostConstruct`	Executes setup or transformation methods after fields inject.
`@Default`	Assigns fallback values if a property is missing.

All Annotaion and properties

Core Model Setup

- `@Model` : **Declares a Java class or interface as a Sling Model.**
  - *Attributes:*
    - `adaptables` : Specifies what this model can be adapted from (e.g., `Resource.class` , `SlingHttpServletRequest.class`). **(Required)**
    - `adapters` : Registers the model under a specific interface or class.
    - `defaultInjectionStrategy` : Sets whether fields are required or optional by default (`DefaultInjectionStrategy.REQUIRED` or `OPTIONAL`).

Injection Annotations

These annotations inject data into your model's fields or constructor parameters.

- `@ValueMapValue` : **Injects properties from the resource's ValueMap** (e.g., JCR properties like `jcr:title`).
  - *Attributes:* `name` (if the JCR property name differs from the variable name).
- `@SlingObject` : **Injects standard Sling toolkit objects.** Works for `ResourceResolver` , `Resource` , `SlingHttpServletRequest` , and `SlingHttpServletResponse` .
- `@ScriptVariable` : **Injects scripting variables available in HTL/JSP scripts** (e.g., `pageManager` , `currentPage` , `properties` , `designer`).
- `@OSGiService` : **Injects an OSGi service** registered in the AEM environment.
- `@RequestAttribute` : **Injects an attribute from the current ServletRequest.**
- `@ResourcePath` : **Injects a resource or a path string** pointing to another resource.
- `@ChildResource` : **Injects a child resource** of the current adaptable resource.
- `@Self` : **Injects the adaptable object itself** (e.g., if adapted from a request, injects the request).

Injection Modifiers

- `@Required` : **Fails model creation if the injected element is missing/null.** This is the default behavior unless `defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL` is set on the `@Model` .
- `@Optional` : **Allows the model to instantiate even if the injected element is missing/null.**
- `@Named` : **Overrides the default naming strategy.** Useful when the JCR property name contains special characters (like `sling:resourceType`) that are invalid as Java variable names.
- `@Default` : **Provides fallback default values** if the injected property is missing. Supports primitives, Strings, and arrays.

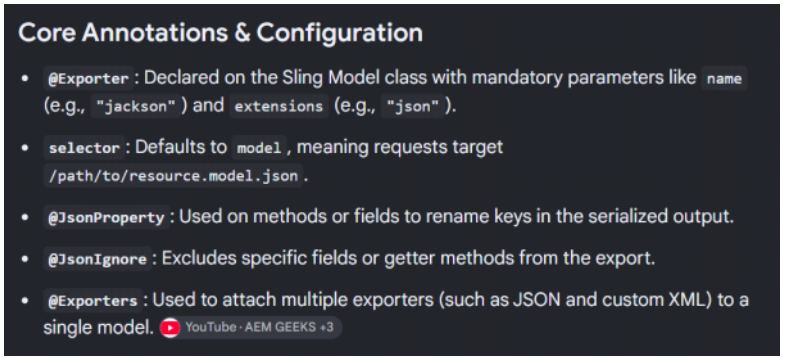
Lifecycle Methods

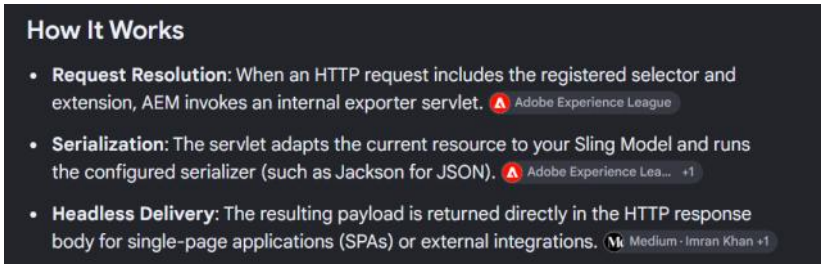
- `@PostConstruct` : **Executes a method immediately after all injections are complete.** Use this for initialization logic, calculations, or data formatting.

Annotations:
@Model -
(adaptable = resouce.class) -> tells this model is mapped to CRX resource , on whatever component we are using this sling model to that resource
(resourceType = 'mysite/components/mylearning) -> optional we can use this or mention directly in the component.html page
(defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL)->means that all injected fields (like @ValueMapValue, @ChildResource, or services) are treated as optional by default. Missing properties will inject null instead of causing model creation to fail

@PostConstruct - method wil be called automatically without creating object of that class
Only 1 method should have this annotation
Method name can be anything eg: init()

Sling model exporter:
serializes Sling Models into custom formats like JSON or XML using the `@Exporter` annotation.
It allows components to function as headless web services by appending selectors and extensions (such as `.model.json`) directly to AEM resource URLs





For Json we use jackson
@Exporter(name = "jackson", extensions = "json")

Hands-on

Creatinga component in intellij using sling model

Write a sling model
Crete a component
Write emp.html
Cq:dialog

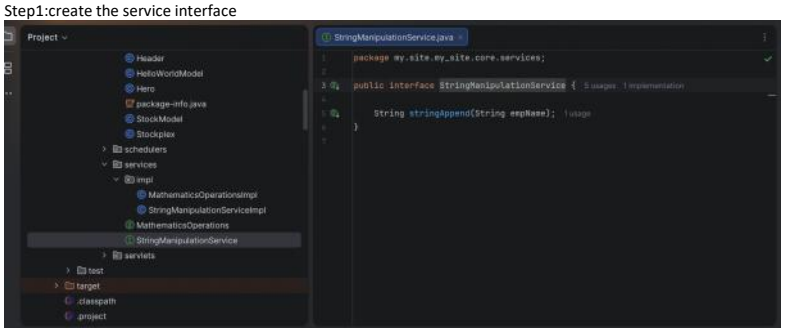
Session -17
Osgi services and components

Always create object of interface class not the interface implementation class(this method is called programminng towards interface class)

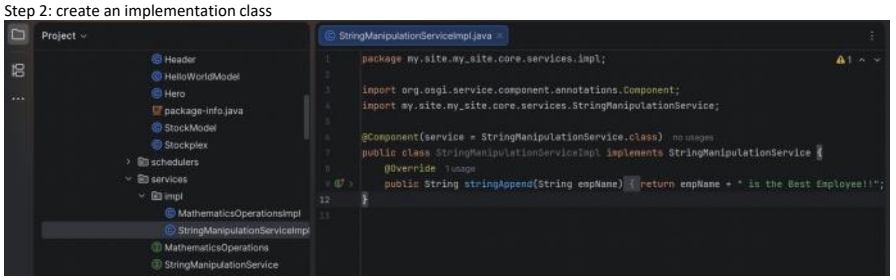
Hands-on

Exercise 1:Create stringManipulationService

Step1:create the service interface



Step 2: create an implementation class



Step 3: call the service inside sling model

```
@Inject
private StringManipulationService stringManipulationService;


private String message;


private boolean isTouch;

@PostConstruct
public void init() {
    //Business logic or any other initialization will come here
    message = stringManipulationService.stringAppend(empname);
}
```

Step 4: Modify the html to display the message

Message from company -: ${model.message}

Exercise 2: using one service in another service

Step1:ddd

@osgiservice:

Session 17:

Intorduction to servlet:

Sling Servlets are server-side Java classes used to handle HTTP requests and generate dynamic responses

Core Base Classes

When writing an AEM servlet, you must extend one of two primary base classes provided by the Sling framework: YouTube · Sankham Ma... +1

- `SlingSafeMethodsServlet` : Used for **read-only operations**. It supports standard idempotent HTTP methods like `GET` , `HEAD` , and `OPTIONS` . Medium
- `SlingAllMethodsServlet` : Used for **read-write operations**. It extends `SlingSafeMethodsServlet` and adds support for data-modifying HTTP methods like `POST` , `PUT` , and `DELETE` . Medium · Imran Khan +1

Registration Types

AEM servlets are registered as OSGi services. There are two main ways to map a servlet to an incoming request: Adobe Experience Lea... +1

1. Resource Type-Based Servlets (Highly Recommended)

These servlets are bound to a specific `sling:resourceType` . When a user requests a JCR content page that utilizes that resource type, the Sling engine invokes this servlet. Medium · Imran Khan +2

- **Security Advantages:** They adhere directly to JCR repository permissions (ACLs). If a user does not have permission to view that resource, they cannot execute the servlet. YouTube · AEM GEEKS +2
- **Configuration:** You can restrict invocation by specifying methods, extensions (e.g., `.json` , `.html`), and selectors. Medium · Imran Khan +1

2. Path-Based Servlets

These servlets are bound directly to one or more absolute URL paths (e.g., `/bin/my-endpoint`). Adobe Experience Lea... +1

- **Security Risks:** They bypass standard JCR repository permission controls. Anyone with access to the URL can trigger the logic, which requires developers to manually implement strict custom permission filters or manage dispatcher rules. YouTube · AEM GEEKS
- **Best Practice:** They should generally be restricted to internal utility tasks or scenarios where an explicit, static endpoint is required. LinkedIn · Suresh Dhuli... +1

Comparing both

Summary Comparison

Attribute	Resource Type-Based Servlet	Path-Based Servlet
Binding Mechanism	Tied to a specific JCR component/resource type	Tied to an explicit URL endpoint
Security Mechanism	Automatic JCR ACL access controls	Manual implementation required
URL Flexibility	Leverages selectors and extensions dynamically	Hardcoded to specific string paths
AEM Best Practice	Preferred approach for application logic	Used sparingly for generic, path-bound tasks

Resource type servlet

Whenever we have to register a servlet via resource we need to provide below 4 things in combination

1. **Resource Type**
2. **Selector**
3. **Extension**
4. **Method type**

Eg:

```
@Component(service = Servlet.class,
```

```
@Component(service = Servlet.class,
        property = {
        "sling.servlet.extensions=json",
        "sling.servlet.selectors=mathsoperations",
        "sling.servlet.methods=" + HttpConstants.METHOD_GET,
        "sling.servlet.resourceTypes=/apps/mysite/components/page" })
public class GetMathematicsOperationsServlet extends SlingSafeMethodsServlet {
```

Path based servlet

Whenever we have to register a servlet via path we just need to provide the dummy path

Eg:

```
@Component(service = Servlet.class, property = {
        Constants.SERVICE_DESCRIPTION + "=JSON path based servlet",
        "sling.servlet.methods=" + HttpConstants.METHOD_GET,
        "sling.servlet.paths=" + "/bin/abc" })
public class PathBasedServlet extends SlingSafeMethodsServlet {
```

Here "sling.servlet.paths=" + "/bin/pathbasedexample"

Exercise 1:

Exercise 2:

Session 18 : Logging and Debugging

Log files In AEM you can see log files for your server in a folder named as logs under quick start folder.
If you go to this location there will be already some out of the log files located which are

1. request.log
2. error.log
3. access.log
4. audit.log
5. stderr.log
6. stdout.log

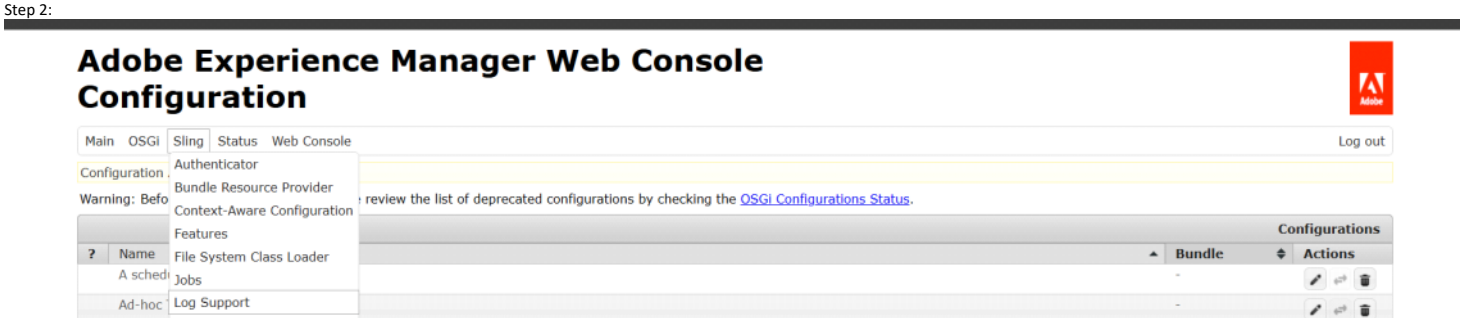
Here most import log file for a developer is error.log file. This file stores all the logs which comes from the code (either from product code or custom code).

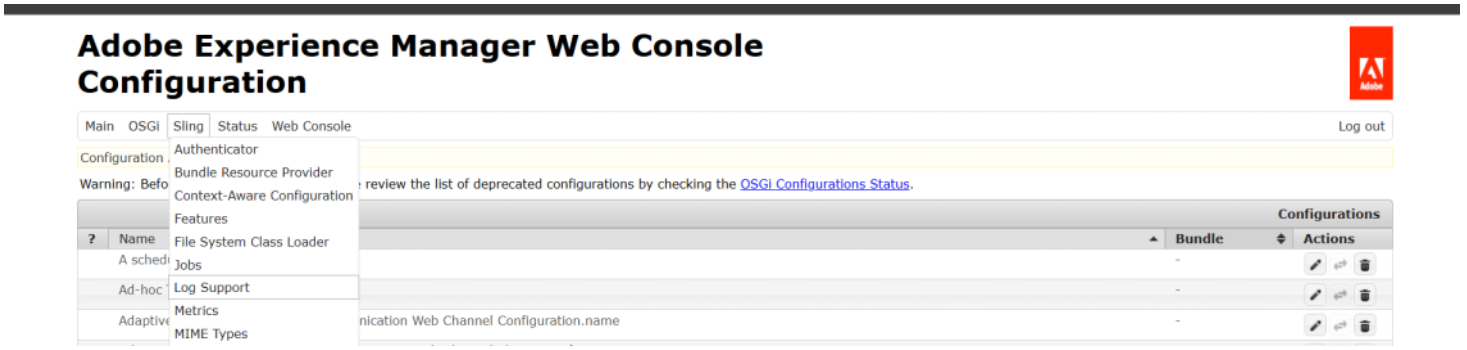
Using log files we can also debug our code in higher environments. These log files works as a alarm system for a running project.

Hands-on:
1.Create Custom Log Files

Step 1:
go to http://localhost:4502/system/console/configMgr

Step 2:





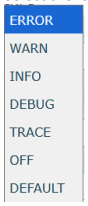
From sling menu select Log Support

Step 3:
Click add new logger



Step 4 :
Give log file name
logs\mani.log

Select the log level according to which it will log the above level log's



If INFO is selected it will log error,warn and info
If error is selected it will only log error

For now will select INFO so will get log of ERROR,WARN and INFO

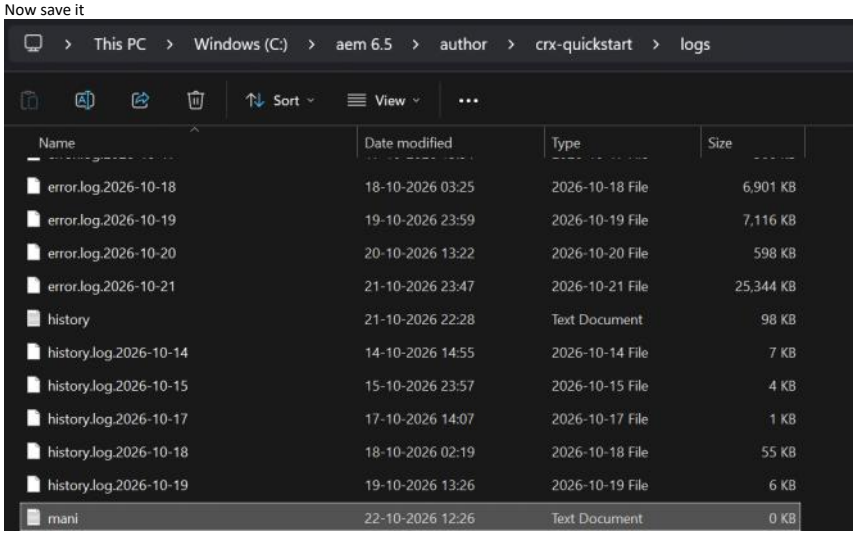
Check the additive checkbox

Enter the package name (project) for which we need to log the errors

my.site.my_site.core



Now save it

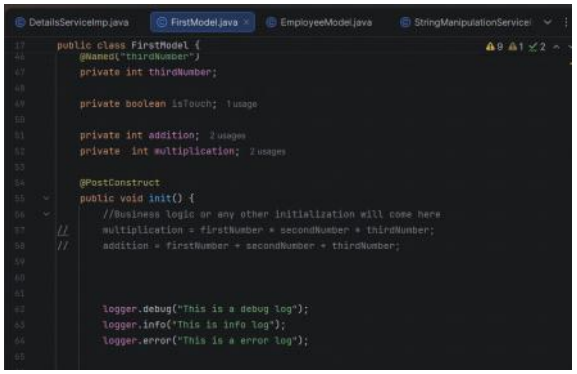


Step 5:

In the model or anywhere use the logger

For now use in model

private final Logger logger = LoggerFactory.*getLogger*(getClass());



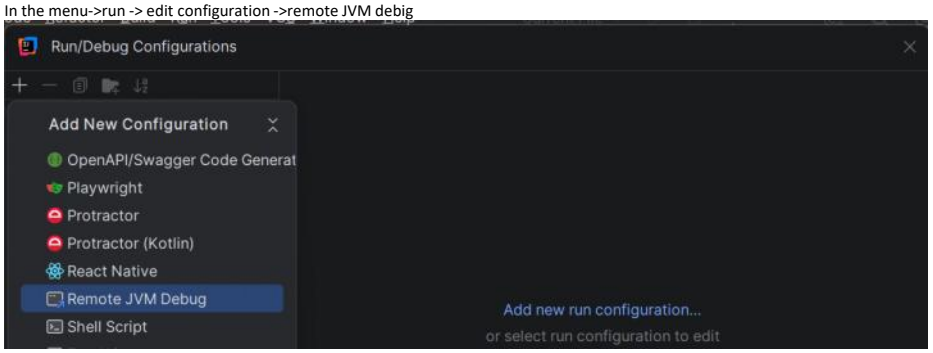
Debugging
Running AEM in Debug mode Continuation of session 17

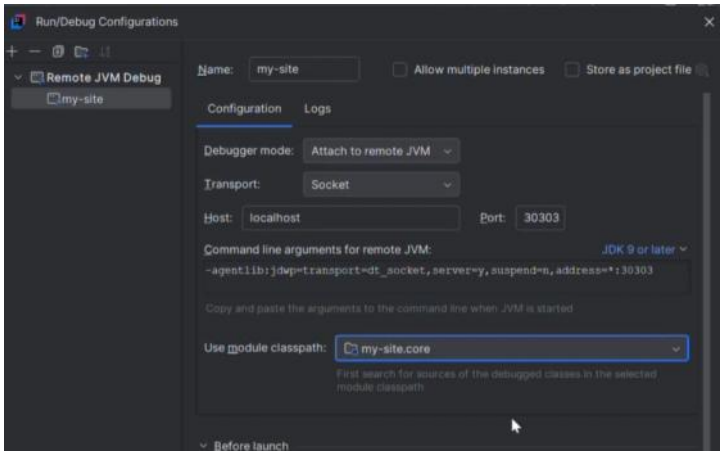
To Do the remote debugging in AEM we have to do below two steps
1. Start your server in debug mode
2. Connect your IDE with the debugging session
Once IDE is connected with AEM server's debugging session then you can start your remote debugging.

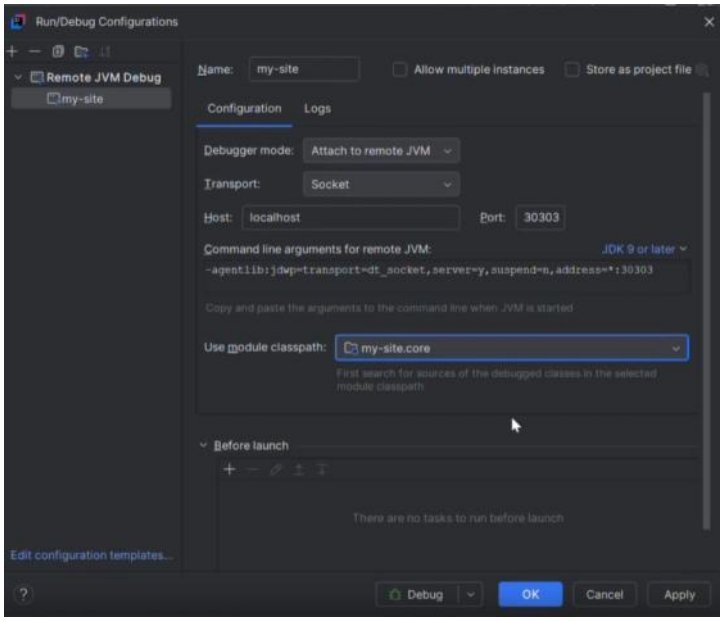
Command to start the AEM server in debug mode use below command stop the author server and run with this
Step1:

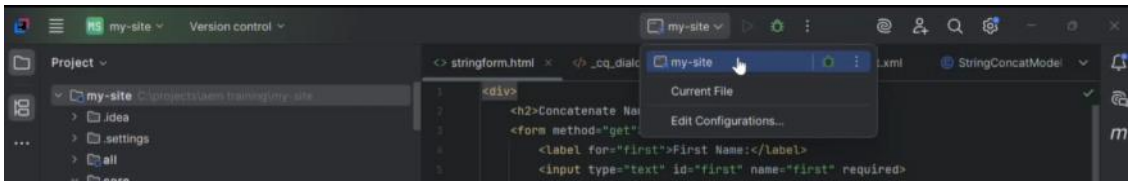
java -Xdebug -Xnoagent -Djava.compiler=NONE -Xrunjdwp:transport=dt_socket,server=y,suspend=n,address=30303 -XX:+HeapDumpOnOutOfMemoryError -Xmx1024m -Dorg.apache.sling.commons.log.level=INFO -jar aem-author-p4502.jar - p 4502 -verbose -nofork

Step 2:
Go to intellij
In the menu->run -> edit configuration ->remote JVM debig







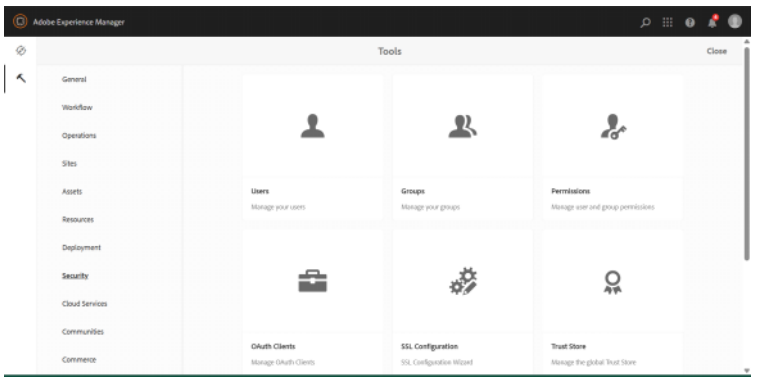


Click on the bug icon beside the my-site

Then we can point the line where we want the debugger to stop

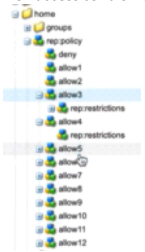
Session 19: users, groups , permissions etc

AEM->toots>security->



Old classic user admin interface -> http://localhost:4502/useradmin

ACL-access control list



Inside rep:policy

Complete the self learinig (session 19 userguid)

Session 15- Custamizing AEM UI
1.overlaying
2.overriding (sling resource merger)

Always sling resource merger is best

Properties

The resource merger provides the following properties:

- `sling:hideProperties` (`String` or `String[]`)

  Specifies the property, or list of properties, to hide.

  The wildcard * hides all.

- `sling:hideResource` (`Boolean`)

  Indicates whether the resources should be completely hidden, including its children.

- `sling:hideChildren` (`String` or `String[]`)

  Contains the child node, or list of child nodes, to hide. The properties of the node will be maintained.

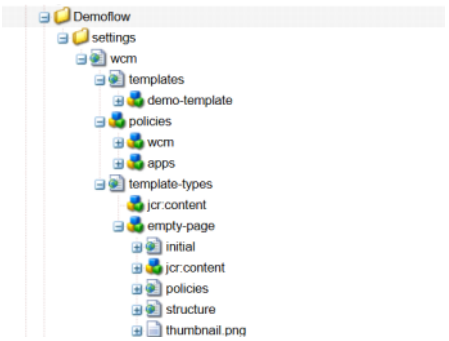
  The wildcard * hides all.

- `sling:orderBefore` (`String`)

  Contains the name of the sibling node that the current node should be positioned in front of.

Hands-on
Session 15
4 exercises

Conf



For editable templated - template and page component have dynamic connection

Template and template type are static connection - full folder structutre(init,poli,struc) will to copied to template when it is created

Modes of Editable templates
-initial = defines the intial content to be displayed on page
-structure = defines the fixed layout and components
-policy = defines the rules and cofig for components
-layout = enable to edit the size of the componet and diff view

Experience Fragment
Content Fragment
Layout container